

TECNOLOGÍAS DE LA INFORMACIÓN EN LINEA INGENIERÍA DE SOFTWARE

3 créditos



Profesor Autor:

Ing. Freddy Malla García, Mtr.

Titulaciones	Semestre
INGENIERÍA DE SOFTWARE	SEXTO

Tutorías: El profesor asignado publicará en el entorno virtual de aprendizaje online.utm.edu.ec, y sus horarios de conferencias se indicarán en la sección **CRONOGRAMA**.

PERIODO ABRIL - AGOSTO 2026



Contenido

UNIDAD III MODELADO DE REQUERIMIENTOS DESDE EL ENFOQUE ORIENTADO A OBJETOS	1
1. Modelados.....	1
1.1. ANÁLISIS DE LOS REQUERIMIENTOS	1
1.2. CONCEPTOS DE MODELADO DE DATOS.....	2
1.2.1. Objetos de datos.....	2
1.2.2. Atributos de los datos	4
1.2.3. Relaciones.	4
2. Modelado basado en escenarios: Casos de uso.	5
2.1. Creación de un caso preliminar de uso	6
2.2. Símbolos de los casos de uso	7
2.3. Relaciones de los casos de uso	9
2.4. Escritura de un caso de uso formal	11
2.5. Mejora de un caso de uso preliminar.....	13
2.6. Desarrollo de diagramas de casos de uso	16
2.7. Desarrollo de escenarios de casos de uso.....	19
El Lenguaje Unificado de Modelado (UML).....	19
3. MODELOS UML QUE PROPORCIONA EL CASO DE USO:	21
3.1. Desarrollo de un diagrama de actividades	21
3.2. Diagramas de canal (swimlane).....	22
4. MODELADO BASADO EN CLASES:	24
4.1. Identificación de las clases de análisis.....	24
4.2. Especificación de atributos.	28
4.3. Definición de las operaciones.	30
5. MODELADO CLASE-RESPONSABILIDAD-COLABORADOR (CRC).....	31

5.1. Modelo de comportamiento:	38
5.2. Identificar los eventos con el caso de uso.....	39
5.3. Representaciones de estado	39
5.4. Diagrama de estado.....	40
5.5. Diagrama de secuencia.	41
6. MODELADO DE REQUERIMIENTOS PARA WEBAPPS	43
6.1. ¿Cuánto análisis es suficiente?	43
6.2. Entrada del modelado de los requerimientos	44
6.3. Salida del modelado de los requerimientos.....	45
6.4. Modelo del contenido de las webapps.....	46
6.5. Modelo de la interacción para webapps	48
6.6. Modelo funcional para las webapps	49
6.7. Modelos de configuración para las webapps	50
BIBLIOGRAFÍA	51

Índice de Ilustración

Ilustración 1: Representación tabular de objetos de datos	3
Ilustración 2: Relaciones entre objetos de datos	5
Ilustración 3: Algunos componentes de los diagramas de caso de uso.....	9
Ilustración 4: Ej. casos de uso y relaciones de comportamiento para la inscripción de estudiantes.....	11
Ilustración 5: Caso de uso de AVC-MVC	12
Ilustración 6: Diagrama de caso de uso preliminar para el sistema CasaSegura	13
Ilustración 7: Diagrama de caso de uso - sistema utilizado para planear una conferencia.....	18
Ilustración 8: UML facilita la comunicación entre los miembros del equipo de desarrollo	21

Ilustración 9: Diagrama de actividades para la función Acceder a la vigilancia con cámaras por internet, mostrar vistas de cámaras.....	22
Ilustración 10: Diagrama de canal para la función Acceder a la vigilancia con cámaras por internet, mostrar vistas de cámaras.....	23
Ilustración 11: Función de seguridad CasaSegura, Clases potenciales	26
Ilustración 12: Clase potencial y número de características que se aplica	28
Ilustración 13: Diagrama de clase para la clase sistema.....	30
Ilustración 14: Diagrama de clase para Plano (véase el análisis en el recuadro)32	
Ilustración 15: Modelo de tarjeta CRC índice IGURA 6.11	32
Ilustración 16: Una clase agregada compuesta	36
Ilustración 17: Diagrama de estado para la clase PaneldeControl.....	40
Ilustración 18: Diagrama de secuencia(parcial) para la función CasaSegura.....	42
Ilustración 19: Árbol de datos para el componente Casa SeguraAsegurada.com	48

UNIDAD III MODELADO DE REQUERIMIENTOS DESDE EL ENFOQUE ORIENTADO A OBJETOS

1. Modelados.

A nivel técnico, la ingeniería de software comienza con una serie de tareas de modelado, estos conducen a la identificación de requisitos y representa el diseño que se va a desarrollar. Diseñar y construir programas de computadora es simplemente desafiante, creativo y divertido. De hecho, la creación de software es tan buena que muchos desarrolladores de software quieren avanzar justo antes de tener una comprensión clara de lo que se necesita.

El Modelo de requerimientos es un conjunto de modelos, es la primera representación técnica del sistema (Pressman, 2010).

1.1. ANÁLISIS DE LOS REQUERIMIENTOS

El análisis de requisitos da la determinación de las características operativas del software, en referencia a su interfaz y otros componentes del sistema, y estipula las limitaciones de software. El análisis de necesidades permite construir en base a los requisitos básicos identificados en las tareas de diseño, investigación y negociación, que pertenecen a la parte de los requisitos de ingeniería (Pressman, 2010).

Hacer solicitudes de modelado da como resultado uno o más de los siguientes tipos de modelo: (Pressman, 2010)

- *Modelos basados en el escenario* de los requerimientos desde el punto de vista de distintos “actores” del sistema.
- *Modelos de datos*, muestra el área de información del problema.
- *Modelos orientados a clases*, representan clases orientadas a objetos (atributos y operaciones) y cómo las clases trabajan juntas para cumplir con los requisitos del sistema.
- *Modelos orientados al flujo*, representan los elementos funcionales del sistema y cómo convierten los datos a medida que viajan a través del sistema.

- *Modelos de comportamiento*, muestran cómo se comparten los programas como resultado de "eventos" externos (Pressman, 2010).

Estos modelos proporcionan al diseñador de software información que se convierte en una arquitectura, una interfaz y un diseño de componentes. Finalmente, el modelo de requisitos (y especificaciones de requisitos de software) ofrecen a desarrolladores y clientes los medio para evaluar la calidad después de construir el software (Pressman, 2010).

Modelado basado en escenarios, una técnica que se está volviendo cada vez más popular en la comunidad de ingeniería de software; modelado basado en datos más especializado, especialmente adecuado cuando se necesita construir una aplicación o manipular información compleja y modelo modelado orientado a clases, que representa capas orientada a objetos y la colaboración resultante permite que el sistema funcione (Pressman, 2010).

1.2. CONCEPTOS DE MODELADO DE DATOS

Si los requisitos del programa incluyen la necesidad de crear, expandir o interactuar con base de datos o si es necesario construir y manipular estructuras de datos complejas, el equipo del programa puede optar por crear un modelo de datos como parte de su modelo general de requisitos. El Ingeniero de software o analista define todos los objetos de datos procesados en el sistema, sus relaciones y otra información relacionada para las relaciones. Los diagramas de diagrama entidad-relación (DER) abordan estos aspectos y representan todos los datos recopilados, almacenados, transformados y creados en una aplicación (Pressman, 2010).

1.2.1. Objetos de datos

Un objeto de datos es una representación compleja de información que necesita ser entendida por el software. La información agregada significa que algo tiene muchos atributos o atributos diferentes. Por lo tanto, el ancho (un valor único) no será un objeto de datos válido, pero las dimensiones (incluidas la altura, el ancho y la profundidad) se pueden especificar como un objeto. El objeto de datos puede ser una entidad externa (por ejemplo, cualquier cosa que cree o consumir información), evento (por ejemplo, un informe o pantalla),

evento (por ejemplo, llamada telefónica) o evento (p. ej., alarma), rol (proveedor), unidad organización (como un departamento de contabilidad), una ubicación (como un almacén) o Estructura (por ejemplo, un archivo). Por ejemplo, una persona o un automóvil pueden considerarse objetos de datos donde cada objeto se identifica mediante un conjunto de atributos (Pressman, 2010).

La descripción de un objeto de datos incluye todos sus atributos. El objeto de datos solo contiene datos: internamente no tiene una referencia a las operaciones que realiza sobre los datos. Entonces el objeto de datos se puede representar como una tabla, como se expresa en la ilustración 1. Los encabezados de las tablas reflejan atributos de alguna cosa. En este caso, el automóvil se identifica por su fabricante, modelo, número de serie, tipo de carrocería, color y propietario. Los contenidos de la tabla representan instancias específicas del objeto de datos de la tabla. Por ejemplo, un Chevy Corvette es un ejemplo de objeto de datos de vehículos (Pressman, 2010).

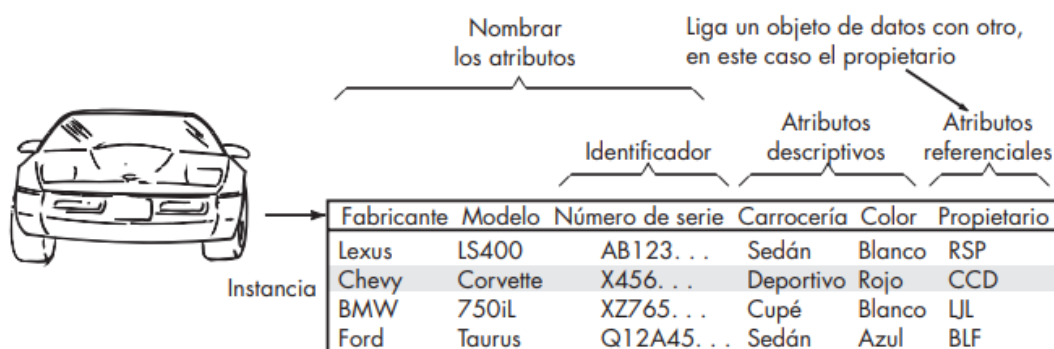


Ilustración 1: Representación tabular de objetos de datos

Fuente: (Pressman, 2010).



Recuerde que. Un objeto de datos es una representación de cualquier información compuesta que se procese en el software.

1.2.2. Atributos de los datos

Los atributos de datos definen las propiedades de un objeto de datos y tienen una de tres propiedades diferentes. Se utilizan para 1) nombrar una instancia de un objeto de datos, 2) describirlo y 3) Apunta a otra instancia en otra tabla. También, debe definirse como identificador uno o más atributos, es decir, el atributo id se convierte en "llave" cuando desee encontrar una instancia de un objeto de datos, en algunos casos, los valores de los identificadores son únicos, aunque esto no es un requisito. En lo que se refiere a objetos de datos auto, un identificador razonable sería el número de serie. Se define el conjunto apropiado de atributos para un objeto de datos dado, se define para comprender el contexto del problema. Los atributos para auto pueden funcionar bien para las aplicaciones que son utilizadas por el departamento de automóviles, pero no serán inútiles para la compañía automotriz necesita crear un programa de control de producción. En este último caso, los atributos del auto también pueden incluir el número de serie, el tipo de carrocería y el color, pero se agregarán más (como código interno, tipo de tracción e indicador de kit de piezas, tipo de caja de cambios, etc.) para que el auto tenga sentido en su contexto control de producción (Pressman, 2010).



Recuerde que. Los atributos nombran a un objeto de datos, describen sus características y, en ciertos casos, hacen referencia a otro objeto.

1.2.3. Relaciones.

Los objetos de datos se relacionan entre sí de diferentes formas. Ten en cuenta dos objetos de datos, personas y auto. Estos objetos están representados por notación simple como se muestra en la ilustración 2 a). Se establece una conexión entre la persona y el auto porque los dos objetos están conectados. Pero, ¿qué es esta relación? Para determinar este papel de las personas (propiedad, en este caso) y auto como parte del programa a

construir. Un conjunto de pares objeto/relación creado para definir relaciones relacionadas. por ejemplo, (Pressman, 2010).

- La persona propietaria de un auto.
- Está asegurado para conducir el auto.

Define las relaciones que tiene y su gestión asegura los vínculos pertinentes entre personas y vehículos, la ilustración 2 b) muestra objeto-relación. Flechas en esta imagen proporcionan información importante sobre la dirección de la relación y, a menudo, reducen ambigüedad o malentendido (Pressman, 2010).

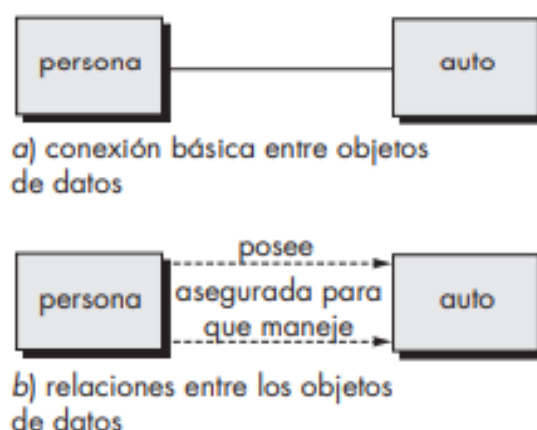


Ilustración 2: Relaciones entre objetos de datos

Fuente: (Pressman, 2010).



Recuerde que. Las relaciones indican la manera en la que los objetos de datos se conectan entre sí.

2. Modelado basado en escenarios: Casos de uso.

Aunque el éxito de cualquier sistema o producto computarizado se mide de muchas maneras, la satisfacción del usuario está en la parte superior de la lista. Al comprender cómo los usuarios finales (y otras partes interesadas) desean interactuar con el sistema, el equipo de software hace un mejor

trabajo, esté preparado para describir completamente los requisitos y realizar análisis significativos y modelos del diseño (Pressman, 2010).

Aunque originalmente se presentó como un diagrama para su uso en UML orientado a objetos, ahora los casos de uso se utilizan independientemente de cómo se desarrolle el sistema. Se puede utilizar como parte SDLC o en el modelado ágil. El modelo de caso de uso describe lo que hace el sistema sin describir cómo lo hace; es decir, es un modelo lógico del sistema. El modelo de caso de uso presenta el sistema desde el punto de vista de un usuario externo, el analista desarrolla casos de uso en asociación con profesionales de negocios que ayudan a determinar los requisitos del sistema. Un modelo de caso de uso proporciona un medio eficaz de comunicación entre el equipo de ventas y el equipo de desarrollo. El modelo de casos de uso desglosa los procesos del sistema en comportamientos, servicios y respuestas (casos de uso) que son trascendentales para los usuarios del sistema. Desde el punto de vista del actor (o usuario), el caso de uso debería producir algo de valor. Por lo tanto, el analista debe identificar lo que es importante para el usuario y debe recordar incluirlo en el diagrama de casos de uso. Por ejemplo, ¿ingresar una contraseña tiene algún valor para el usuario? Esto probablemente debería incluirse si el usuario tiene problemas de seguridad o si son críticos para el éxito del proyecto (Kendall & Kendall, 2011).

Por lo tanto, una solicitud de modelado con UML inicia con escenarios en forma de casos de uso, diagramas de actividades y diagramas de tipos carril de natación (Pressman, 2010).

2.1. Creación de un caso preliminar de uso

Alistair Cockburn describe el caso de uso como un "contrato de comportamiento". El "contrato" define como un actor usa un sistema informático para conseguir un objetivo. Básicamente un caso de uso capta las interacciones que suceden entre productores y consumidores de información y el sistema mismo (Pressman, 2010).

Un caso de uso describe un caso específico en lenguaje simple, desde el punto de vista de un representante particular. Pero, ¿cómo sabes qué escribir

y cuánto escribir sobre ello, describa en detalle la descripción y cómo organizarlo. Estas son las preguntas que se debe responder si los casos de uso son valiosos como herramienta de modelado de requisitos (Pressman, 2010).

¿Sobre qué escribir? Las dos primeras tareas de Ingeniería de Requisitos, concepción e indagación, brindan la información necesaria para empezar a escribir casos de uso (Pressman, 2010).

Reuniones de recolección de requerimientos, DEC y otros mecanismos se utilizan para identificar a los participantes, definir el alcance del problema, definir los objetivos comerciales generales, establecer prioridades, describir todos los requisitos funcionales conocidos y describir los elementos (objetos) que administrará el sistema. Para comenzar a desarrollar un conjunto de casos de uso, se enumeran los trabajos o actividades que realiza un actor en particular. Se obtiene de la lista de funciones requeridas del sistema, mediante discusión con los participantes o mediante evaluación (Pressman, 2010).

2.2. Símbolos de los casos de uso

El diagrama de caso de uso incluye los símbolos del actor y del caso de uso, así como las líneas conectoras, los actores son similares a entidades externas; existen fuera del sistema. El término actor se refiere a un papel para usuarios del sistema. Por ejemplo, un actor puede ser un empleado, pero también puede ser clientes en la tienda de la empresa, incluso si es la misma persona en el mundo real, se expresa como dos iconos diferentes en el diagrama de casos de uso, porque la persona interactúa con el sistema en diferentes roles (Kendall & Kendall, 2011).

El actor está fuera del sistema e interactúa con él de cierta manera. Un representante puede ser una persona, otro sistema o dispositivo como un teclado o conexión a Internet. Los actores pueden iniciar una instancia de un caso de uso, un actor puede interactuar con uno o más casos de uso; un caso de uso potencial relacionado con o más repeticiones, los representantes se pueden dividir en varios grupos (Kendall & Kendall, 2011).

Actores principales proporcionan datos o reciben información del sistema, algunos usuarios interactúan directamente con el sistema (actores del sistema), pero los actores interactúan con personas de negocios que también pueden ser empresarios que no interactúan directamente con el sistema, pero comparten de una forma u otra. Los actores principales son importantes porque son usuarios del sistema y pueden proporcionar detalles sobre lo que debe hacer el caso de uso. También pueden proporcionar una lista de objetivos y prioridades. Los actores de apoyo (también llamados actores secundarios) que ayudan a mantener el sistema en funcionamiento o brindan otros servicios; son personas que operan en el departamento como soporte técnico, analistas, programadores, entre otros, (Kendall & Kendall, 2011).

A veces es apropiado crear perfiles de actores que enumeren a los actores, sus antecedentes y habilidades en forma de tabla. Esto puede ser útil para comprender cómo interactúa el actor con el sistema. por ejemplo, el perfil del Profesional de Tramitación de Pedidos será: “Usuario habitual del software familiarizado con las funciones menores, excepciones de órdenes y personalización de pedidos. También debe hacer una lista de actores con sus objetivos y prioridades (Kendall & Kendall, 2011).

Cada objetivo se puede cambiar en caso de uso; los casos de uso brindan a los desarrolladores una descripción general de lo que quieren los usuarios sin detalles técnicos o de implementación. Podemos pensar en un caso de uso como una serie de transacciones en el sistema, el modelo de casos de uso se fundamenta en las interacciones y relaciones de los casos de uso individuales (Kendall & Kendall, 2011).

El caso de uso siempre describe tres cosas: el actor inicial del evento y el evento desencadena un caso de uso y los casos de uso realizan las acciones desencadenadas por el evento. En caso de uso, un actor que aplica el sistema inicia un evento que conduce una serie de interacciones relacionadas en el sistema. Los casos de uso se utilizan para registrar una sola transacción o evento (Kendall & Kendall, 2011).

El evento es una entrada en el sistema que ocurre en un momento y lugar específicos y obliga al sistema a hacer algo. Es mejor crear pocos casos de uso que crear demasiados casos de uso, normalmente no se incluyen consultas e informes; 20 casos de uso es suficiente para un sistema grande (y no más de 40 o 50). También los casos de uso se pueden anidar, si es necesario (Kendall & Kendall, 2011).

Ciertos casos de uso usan el verbo administrar para agrupar casos de uso, incluso se pueden agregar, eliminar y reemplazar con un diagrama de caso de uso de bajo nivel. Puede incluir un caso de uso en varios diagramas, pero el caso de uso real se define solo una vez en el diagrama repositorio. Un nombre de caso de uso consta de un verbo y un sustantivo (Kendall & Kendall, 2011).

2.3. Relaciones de los casos de uso

Las relaciones activas se denominan relaciones de comportamiento y se utilizan especialmente en diagramas de casos de uso. Existen cuatro tipos básicos de relaciones comportamiento: comunica, incluye, extiende y generaliza, tenga en cuenta que todos estos términos son verbos. La ilustración 3, muestra las flechas y líneas que se usan para trazar cada uno de los cuatro tipos de relaciones de comportamiento. Ahora describiremos estas cuatro relaciones.

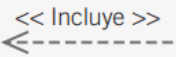
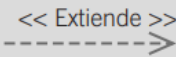
Relación	Símbolo	Significado
Comunica		Para conectar un actor con un caso de uso se utiliza una línea sin puntas de flecha.
Incluye		Un caso de uso contiene un comportamiento común para más de un caso de uso. La flecha apunta al caso de uso común.
Extiende		Un caso de uso distinto maneja las excepciones del caso de uso básico. La flecha apunta del caso de uso extendido al básico.
Generaliza		Una "cosa" de UML es más general que otra "cosa". La flecha apunta a la "cosa" general.

Ilustración 3: Algunos componentes de los diagramas de caso de uso
Fuente: (Kendall & Kendall, 2011)

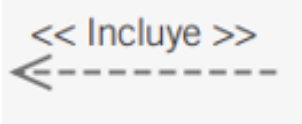
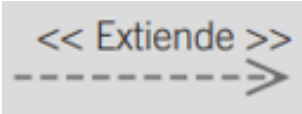
Componentes de los diagramas de caso de uso	Descripción
COMUNICACIÓN 	Esta relación de comportamiento se emplea para conectar un actor con un caso de uso. Tenga presente que la tarea del caso de uso es proporcionar algún tipo de resultado que sea conveniente para el actor del sistema. Entonces, es importante documentar estas relaciones entre actores y casos de uso. Por ejemplo, un estudiante llama para inscribirse en una clase, los diagramas de casos de uso de la ilustración 4 muestra algunos componentes de la ejemplo de matriculación de estudiantes.
INCLUSIÓN 	Esta relación (también conocida como relación de uso) describe una situación en la que los casos de uso tienen un comportamiento común a muchos otros casos de uso. Es decir, el caso de uso común se introduce en otros casos de uso. La flecha punteada indica el caso de uso común que indica la relación entre inclusión. Por ejemplo un caso de uso de pagar la tasa de matrícula que figura en el registro del curso y la gestión del alojamiento, debiendo en ambos casos el propio estudiante abonar las tasas, algunos casos puede usar esto; las flechas indican casos de uso común.
EXTENSIÓN 	Esta relación refiere una situación en la que un caso de uso tiene un comportamiento permisivo del nuevo caso de uso de manejar la diferencia o excepción al caso de uso base. Por ejemplo, caso de uso extendido de cobertura de salud para estudiantes extiende el caso de uso del pago básico. La flecha va desde casos de uso extendidos hasta casos de uso básicos.
GENERALIZACIÓN 	Esta relación indica que una cosa es más común que otra. Esta relación puede existir entre actores o dos casos de uso. Por ejemplo, un estudiante a tiempo parcial generalmente se considera un estudiante de pregrado, asimismo, algunos de los empleados de la universidad son profesores. La flecha apunta a la cosa general.

Tabla 1: Componentes de los diagramas de caso de uso

Fuente: (Kendall & Kendall, 2011)

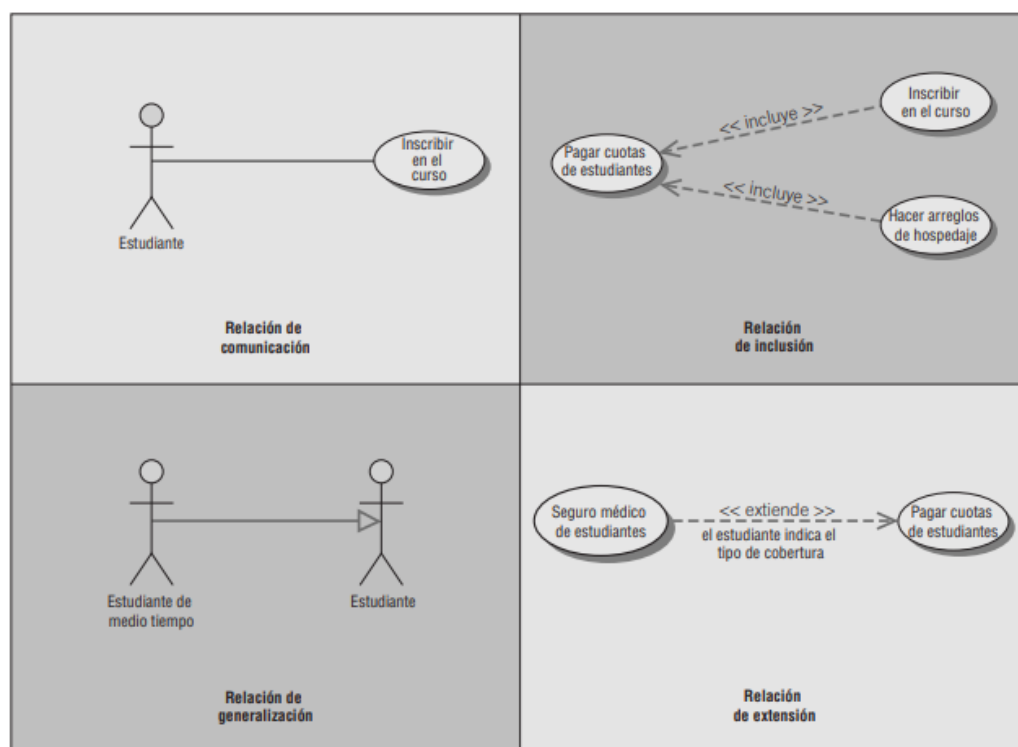


Ilustración 4: Ej. casos de uso y relaciones de comportamiento para la inscripción de estudiantes

Fuente: (Kendall & Kendall, 2011)

2.4. Escritura de un caso de uso formal

A veces, los casos de uso informales son suficientes para modelar los requisitos. Sin embargo, cuando el caso de uso incluye una actividad importante o cuando se describe un conjunto complejo de pasos con una gran cantidad de valores atípicos de excepciones, se recomienda un enfoque más formal. El caso de uso de AVC-MVC que se muestra en la ilustración 5, sigue el texto general para casos de uso formales. La intención en contexto define el alcance general del caso de uso, las condiciones previas describen lo que se sabe que es cierto antes de iniciar un caso de uso. Un disparador (o trigger) que define un evento o condición de "comienzo de un caso de uso" [Coc01b]. El escenario enumera las acciones específicas solicitadas por el actor y las respuestas del sistema adecuado. Las excepciones definen situaciones descubiertas durante la optimización del caso de uso inicia, los encabezados adicionales pueden o no estar incluidos y se explican por sí mismos razonablemente (Pressman, 2010).

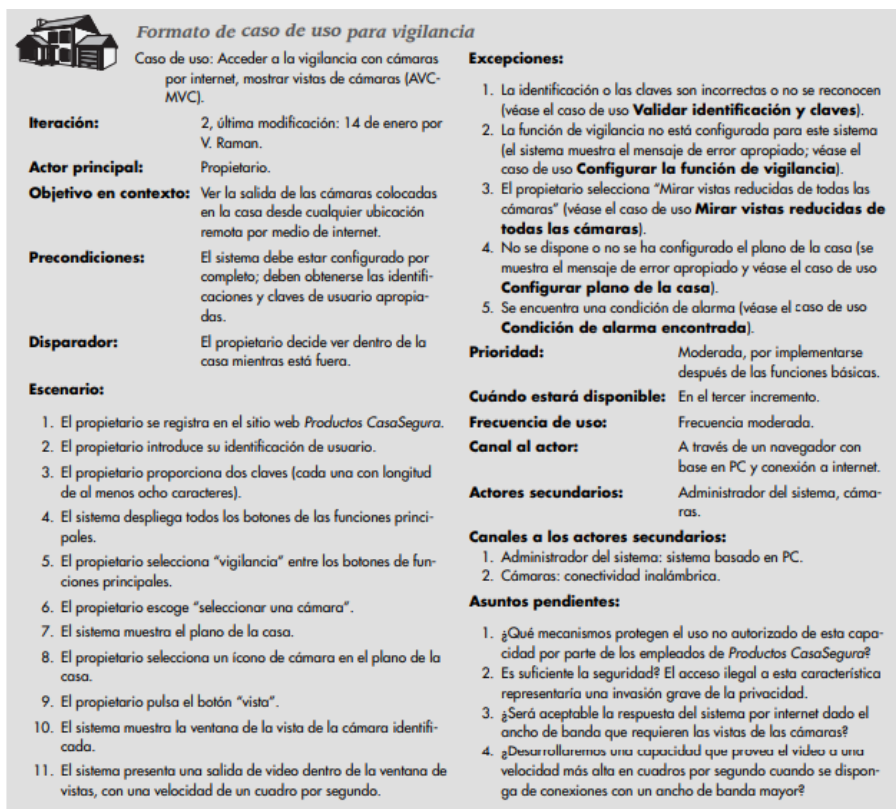


Ilustración 5: Caso de uso de AVC-MVC

Fuente: (Pressman, 2010)

En diversos casos, no es necesario crear una representación gráfica de un escenario de uso, sin embargo, la representación gráfica es fácil de entender, en particular cuando el escenario es complicado. Como se mencionó anteriormente en este libro, el UML es capaz de crear diagramas de casos de uso. La ilustración 6, muestra un diagrama aproximado de un caso de uso para productos CasaSegura. Cada caso de uso está representado por un óvalo.

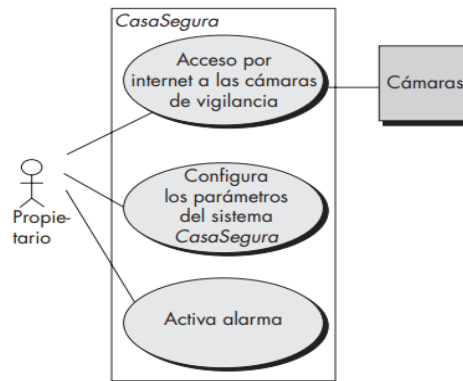


Ilustración 6: Diagrama de caso de uso preliminar para el sistema CasaSegura

Fuente: (Pressman, 2010)

Cada notación de modelado tiene sus limitaciones y la notación de casos de uso no es una excepción. Como cualquier otra forma de descripción escrita, un caso de uso es tan bueno como parecen sus autores. Si la definición no es clara, el caso de uso será confuso o ambiguo. Un caso de uso se centra en los requisitos funcionales y de comportamiento, y en general no cumple con los requisitos disfuncionales. Para situaciones en las que se requiere que el modelo sea lo suficientemente detallado y específico (p. ej., normas críticas de seguridad), un caso de uso único puede no ser suficiente.

Sin embargo, el modelo basado en escenarios es adecuado para las situaciones a las que se enfrentará un ingeniero de software. Si todo va bien, el caso de uso ofrece grandes ventajas como herramienta de modelado (Pressman, 2010).

2.5. Mejora de un caso de uso preliminar

Para comprender completamente la funcionalidad descrita en un caso de uso, es necesario describir las interacciones alternativas. Luego, cada paso en el escenario principal es evaluado e incrementado planteando las siguientes preguntas [Sch98a]:(Pressman, 2010)

- ¿Puede el actor tomar más medidas en este momento?
- ¿Existe la posibilidad de que el actor esté sufriendo una condición de error en este punto? Si este es el caso, ¿Cuál sería?

• En este punto, ¿puede el actor encontrar un comportamiento diferente (por ejemplo: ser llamado por un evento fuera del control del actor)? En este caso, ¿cuál sería?

Las contestaciones a estas interrogaciones crean una variedad de situaciones que son parte del caso de uso original, pero representan los comportamientos alternativos (Pressman, 2010).

Caso de uso: acceder a la vigilancia con cámaras por internet, mostrar vistas de cámaras (AVC-MVC) Actor: propietario

1. El propietario accede al sitio web Productos CasaSegura.
2. El propietario introduce su identificación de usuario.
3. El propietario escribe dos claves (cada una de al menos ocho caracteres de longitud).
4. El sistema muestra los botones de todas las funciones principales.
5. El propietario selecciona "vigilancia" de los botones de las funciones principales.
6. El propietario elige "seleccionar una cámara".
7. El sistema presenta el plano de la casa.
8. El propietario escoge el ícono de una cámara en el plano de la casa.
9. El propietario selecciona el botón "vista".
10. El sistema presenta la ventana de vista identificada con la elección de la cámara.
11. El sistema muestra un video dentro de la ventana a velocidad de un cuadro por segundo.

Es importante observar que esta presentación en secuencia no considera interacciones alternativas (la narración fluye con más libertad y representa varias alternativas). Los casos de este tipo en ocasiones se denominan escenarios primarios [Sch98a]

Tabla 2: Caso de uso: acceder a la vigilancia con cámaras por internet

Fuente: (Pressman, 2010)

Por ejemplo, considere los pasos 6 y 7 del escenario principal descrito:

6. El propietario elige "Selección de cámara".
7. El sistema muestra el plano de planta de la casa.

¿Puede el actor realizar otro trabajo en este punto? La respuesta es sí. "A medida que se continúa analizando la historia, los actores pueden optar por ver escenas de todas las cámaras simultáneamente. Entonces, un escenario

secundario sería "observar imágenes de todas las cámaras". ¿Está el actor enfrentando un error hasta este punto? Cualquier número de condiciones de error puede ocurrir cuando se usa un sistema computarizado. En esto contexto, sólo los términos que probablemente resulten directamente de la acción descrita en el paso 6 o 7; la respuesta sigue siendo sí".

Es posible que el plano de planta con iconos de cámara nunca se haya configurado. Por lo tanto, escoger "Seleccionar cámara" da como resultado una condición de error: "No se han seleccionado planos para esta casa". Esta condición de error se convierte en un escenario secundario. En este punto, ¿el actor puede encontrar otro comportamiento (como el comportamiento llamado por un evento fuera del control del actor)? De nuevo, la respuesta es sí." Después de los pasos 6 y 7, el sistema puede encontrar una condición de alarma, esto le dará como resultado, que el sistema muestra un mensaje de advertencia especial (Tipo, Ubicación, Acción del sistema) y proporcionará al actor una serie de opciones pertinentes en función de la naturaleza de advertencia (Pressman, 2010).

Dado que este escenario secundario puede ocurrir en cualquier momento para casi cualquier interacción, no forma parte del caso de uso de AVC-MVC. En cambio, se desarrollará un caso de uso diferente - se cumple la condición de alarma - diferente al que se va a ejecutar y al que se va a referir otros casos, según sea necesario (Pressman, 2010).

Cada condición descrita en los párrafos anteriores se describe como una excepción de caso de uso. Una excepción describe una situación (una condición de falla o una alternativa elegida por actor) que hace que el sistema se comporte de manera ligeramente diferente (Pressman, 2010).

Un individuo Cockburn [Coc01b] recomienda utilizar una sesión de "lluvia de ideas" para conseguir un conjunto razonablemente complejo de excepciones para cada caso de uso. Aparte se han sugerido preguntas generales, también se deben explorar los siguientes aspectos: (Pressman, 2010)

- ¿Hay algún caso en el que se produzca una "función de validación" en este caso de uso?

Esto significa que se llama a la función de afirmación y puede ocurrir una posible condición de error.

- ¿Hay casos en los que la función (o actor) de apoyo no responde adecuadamente? Por ejemplo, una acción del usuario espera una respuesta, pero la función a responder cae.

- ¿El rendimiento deficiente del sistema conduce a acciones inesperadas o inapropiadas?

Por ejemplo, la interfaz web es muy lenta para responder, lo que resulta que el usuario realice múltiples elecciones en el botón de procesamiento. Estas opciones están mal configuradas y al final generan un error (Pressman, 2010).

La lista de extensiones desarrollada como resultado de hacer y responder estas preguntas [Coc01b] debe "racionalizarse" de acuerdo con los siguientes criterios: Una excepción debe describirse en un caso de uso si el programa puede detectarlo y debe manejarlo una vez descubierto. En algunos casos, la excepción estimulará el desarrollo de otro caso de uso (desde el tratamiento de la condición descrita) (Pressman, 2010).

2.6. Desarrollo de diagramas de casos de uso

El caso de uso principal incluye un flujo estándar de eventos que describen el comportamiento estándar del sistema. El caso de uso inicial representa la terminación normal, esperada y exitosa de un caso de uso. Para crear un diagrama de casos de uso, comience pidiéndole al usuario una lista de todo lo que el sistema tiene que hacer por él. Esto se puede hacer a través de entrevistas, durante una sesión de diseño de aplicaciones juntos o a través de otras sesiones dirigidas por grupos. Los analistas también pueden usar sesiones de historias rápidas para desarrollar casos de uso (Kendall & Kendall, 2011).

Tenga en cuenta quién está involucrado en cada caso de uso y a qué responsabilidades o servicios está sujeto el caso de uso proporcionado a los actores u otros sistemas. En las primeras etapas, puede ser parte de un menú extendido en las siguientes etapas de análisis. Utilice los siguientes principios: (Kendall & Kendall, 2011).

1 Revisar las características de la obra e identificar a los actores involucrados (Kendall & Kendall, 2011).

2 Defina eventos de alto nivel y desarrolle casos de uso básicos que describan estos eventos, además de cómo los actores lo inician. Fíjate bien en los papeles de los actores, define todos los casos de uso principales que cada actor puede iniciar, no tienes que presentar los casos de uso con poca o ninguna interacción del usuario (Kendall & Kendall, 2011).

3 Examine cada uno de los principales casos de uso para identificar posibles diferencias de subprocessos en el caso de uso. Con base en este análisis, cree métodos alternativos. Porque en general el flujo de eventos es diferente en cada caso, busque actividades que puedan o no tener éxito. buscar también ramas en la lógica de casos de uso donde se pueden obtener diferentes resultados. Los diagramas de flujo de datos a nivel de contexto pueden ser el punto de partida para un caso de uso. Las entidades externas son actores potenciales, luego revise el flujo de datos para determinar si puede comenzar un caso de uso o puede ser creado por un caso de uso (Kendall & Kendall, 2011).

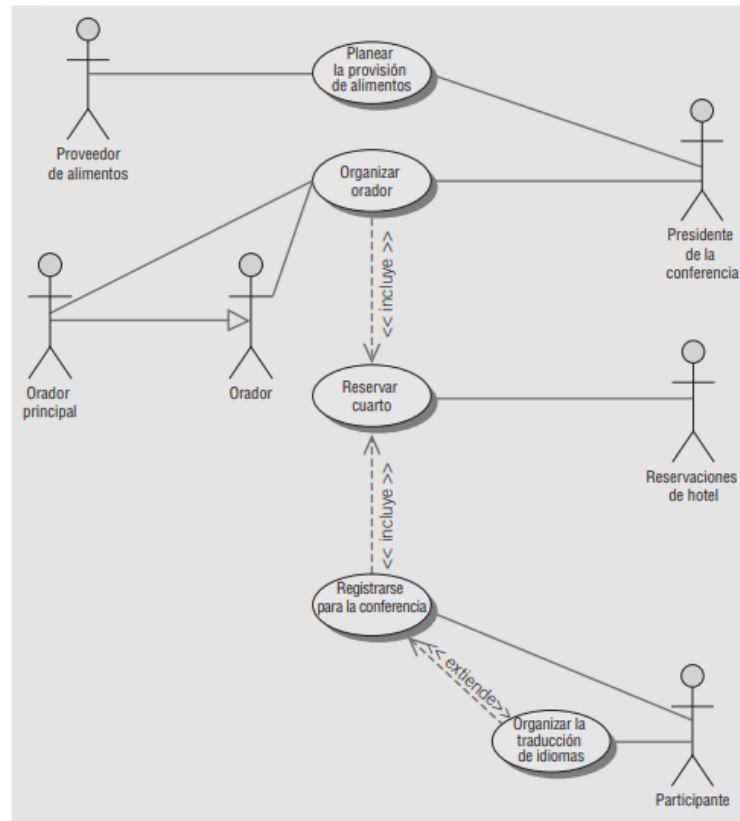


Ilustración 7: Diagrama de caso de uso - sistema utilizado para planear una conferencia

Fuente: (Kendall & Kendall, 2011)

La ilustración 7, muestra un ejemplo de un diagrama de caso de uso que representa un sistema en uso. Fije una fecha para una conferencia. Los actores son el presidente de la conferencia, responsable de planificar y administrar la conferencia, sus participantes, oradores, oradores principales, reservas de hotel y proveedor de alimentos. Los actores representan los roles de los usuarios y los proveedores de alimentos podrían ser del personal del hotel o del servicio de comidas externo. Los anfitriones de conferencias y los servicios de catering ayudan a planificar comidas y fiestas. El Presidente de la conferencia como el Proveedor de alimentos están involucrados en la planeación de las comidas y los banquetes al orador (Kendall & Kendall, 2011).

El Presidente de la conferencia también es responsable de organizar a los oradores. Los participantes se registran para participar en la conferencia. Tenga en cuenta que el caso de uso de Reservar cuarto entrar en una relación integral con los casos de uso Organizar orador y Registrarse para la

conferencia, donde los disertantes y participantes requerirán alojamiento, ya que tanto los oradores como los participantes necesitarán alojamiento. Casos de uso organización de traducción en un idioma ha ampliado el registro de conferencias para todos y que los participantes solicitarán servicios de traducción de idiomas. El actor Orador es una generalización de Orador principal (Kendall & Kendall, 2011).

2.7. Desarrollo de escenarios de casos de uso

Cada caso de uso tiene una descripción. Nos referiremos a la descripción como un caso de uso, el caso de uso principal representa el flujo estándar de eventos en el sistema, y los métodos alternativos describen diferencias en el comportamiento. Use escenarios de casos para describir qué sucede al comprar un artículo que no está disponible, o si la compañía de la tarjeta de crédito rechaza la orden de compra del cliente. No existe un formato estándar para los casos de uso, por lo que cada organización debe definirlos los estándares a incluir. Los casos de uso generalmente se documentan mediante el formulario documentación de casos de uso por parte de la organización, haciendo que los casos de uso sean más legibles y proporcionando información estandarizada para cada caso de uso en el modelo (Kendall & Kendall, 2011).

El Lenguaje Unificado de Modelado (UML)

UML (Unified Modelling Language-Lenguaje de modelado unificado) es un lenguaje de modelado para sistemas de información, es el software más popular y ampliamente utilizado en la actualidad. Se llama "unificado" ya que para crearlo se incorporan tres principales representantes del diseño orientado a objetos: (M. Gómez et al., 2019)

Grady Booch: Desarrolló su propio estilo de análisis y diseño orientado a objetos (M. Gómez et al., 2019).

James Rumbaugh: su blog de diseño orientado a objetos se llama OMT (Tecnología de Modelado de Objetos-Object Modeling Technique) (M. Gómez et al., 2019).

Ivar Jacobson: análisis y diseño orientado a objetivo, creador de casos de uso (M. Gómez et al., 2019).

A mediados de la década de 1990, los tres comenzaron a intercambiar documentos y trabajaron juntos. Obtuvieron una versión UML 1.0, eso es un gran paso en Modelado de Sistemas Objetivos (M. Gómez et al., 2019).

OMG (Object Management Group) es la asociación que fundó y gestiona los estándares tecnológicos orientados a objetivos. En 1997 nació UML cuando es aceptado y aprobado por OMG. Después de pasar por algunas revisadas y mejorada hasta el momento, la última versión de UML es la 2.5, (M. Gómez et al., 2019).

UML es un lenguaje modelado, no una metodología de desarrollo. En la metodología de desarrollo, cada método se especifica en terminología de lenguajes de modelado y procesos de modelado. UML se utiliza como lenguaje gráfico en: (M. Gómez et al., 2019).

- visualizar
- especificar
- construir
- documentar

UML es intuitivo y a través de su sintaxis se modelan diferentes aspectos que permite una mejor explicación y comprensión de un tema o sistema informático. Los elementos UML se utilizan para crear diagramas y no están diseñados exclusivamente para programas orientados a objetos. UML consta de un número de elementos matemáticos que representan diferentes partes de un sistema de software, es decir, consta de varios tipos de diagramas. cada gráfico se utiliza para modelar diferentes partes o perspectivas de un sistema de software. Entonces: UML es un lenguaje de modelado para diferentes aspectos del sistema del programa como se muestra en la ilustración 8, los diagramas UML facilitan la comunicación entre el equipo de desarrollo de software (M. Gómez et al., 2019).



Ilustración 8: UML facilita la comunicación entre los miembros del equipo de desarrollo

Fuente: (M. Gómez et al., 2019)

3. MODELOS UML QUE PROPORCIONA EL CASO DE USO:

Existen situaciones de modelado de requisitos en las que se basa el modelo en texto, incluso es tan simple como un caso de uso, puede que no proporcione información de forma clara y concisa. En tales casos, es posible elegir entre una variedad de modelos UML gráficos (Pressman, 2010).

3.1. Desarrollo de un diagrama de actividades

El diagrama de actividad UML mejora el caso de uso aportando una representación gráfica del flujo de interacción en un escenario dado. El diagrama de actividades es similar a uno de flujo y usa rectángulos redondeados circulares para representar una función particular, flechas para indicar flujo y rombos de decisión para indicar ramificación de decisiones (se nombra cada flecha que sale del rombo) y líneas continuas indican que se están produciendo actividades paralelas. La ilustración 9 muestra el diagrama de actividades para el caso de uso AVC-MVC. Tenga en cuenta que los programas de actividades agregan detalles adicionales que no se mencionan directamente (pero implícito) en el caso de uso. Por ejemplo, un usuario puede hacer solo algunos intentos para ingresar su ID y contraseña. Esto representa

por el rombo de decisión una "Declaración de Repetición de entrada" (Pressman, 2010).

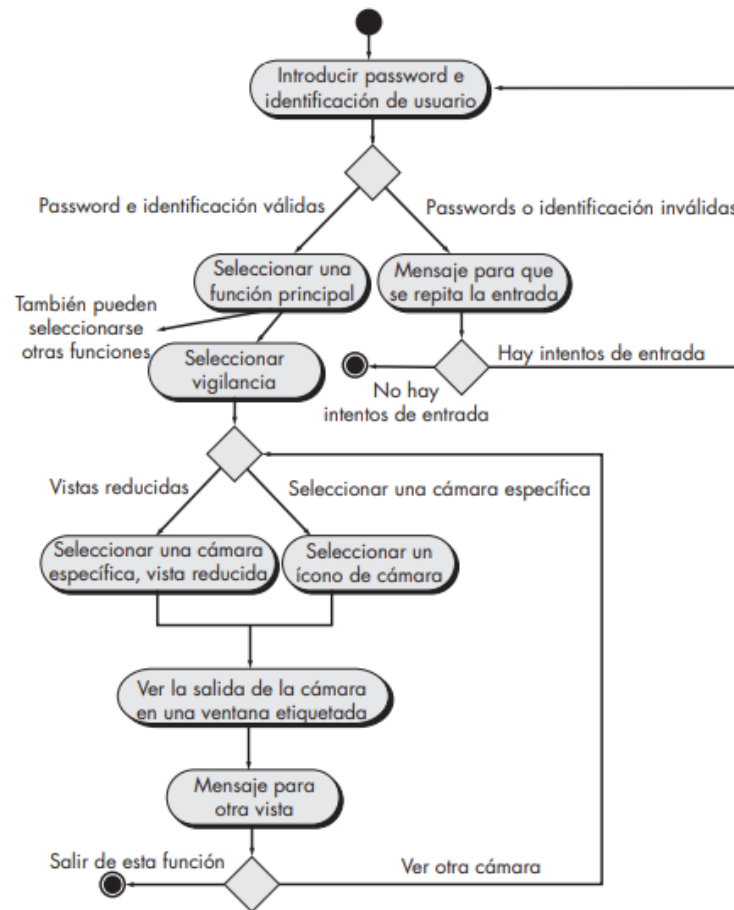


Ilustración 9: Diagrama de actividades para la función Acceder a la vigilancia con cámaras por internet, mostrar vistas de cámaras.

Fuente: (Pressman, 2010).



Recuerde que. Un diagrama de actividades UML representa las acciones y decisiones que ocurren cuando se realiza cierta función.

3.2. Diagramas de canal (swimlane)

El diagrama de canal UML es una variación útil del diagrama de actividad y le permite representar el flujo de actividades descrito en un caso de uso; Al mismo tiempo, se refiere a cualquier actor (si se tiene varias personas involucradas en un caso de uso particular) o una clase de análisis es

responsable de la acción descrita por el rectángulo de actividad(Pressman, 2010).

Las responsabilidades están representadas por segmentos paralelos que dividen el diagrama en verticalmente, como los canales o carriles de una alberca(Pressman, 2010).

Hay tres capas de análisis: propietario, cámara, interfaz que tienen responsabilidad directa o indirectamente en el contexto del diagrama de actividad que se muestra en la ilustración 9(Pressman, 2010)..

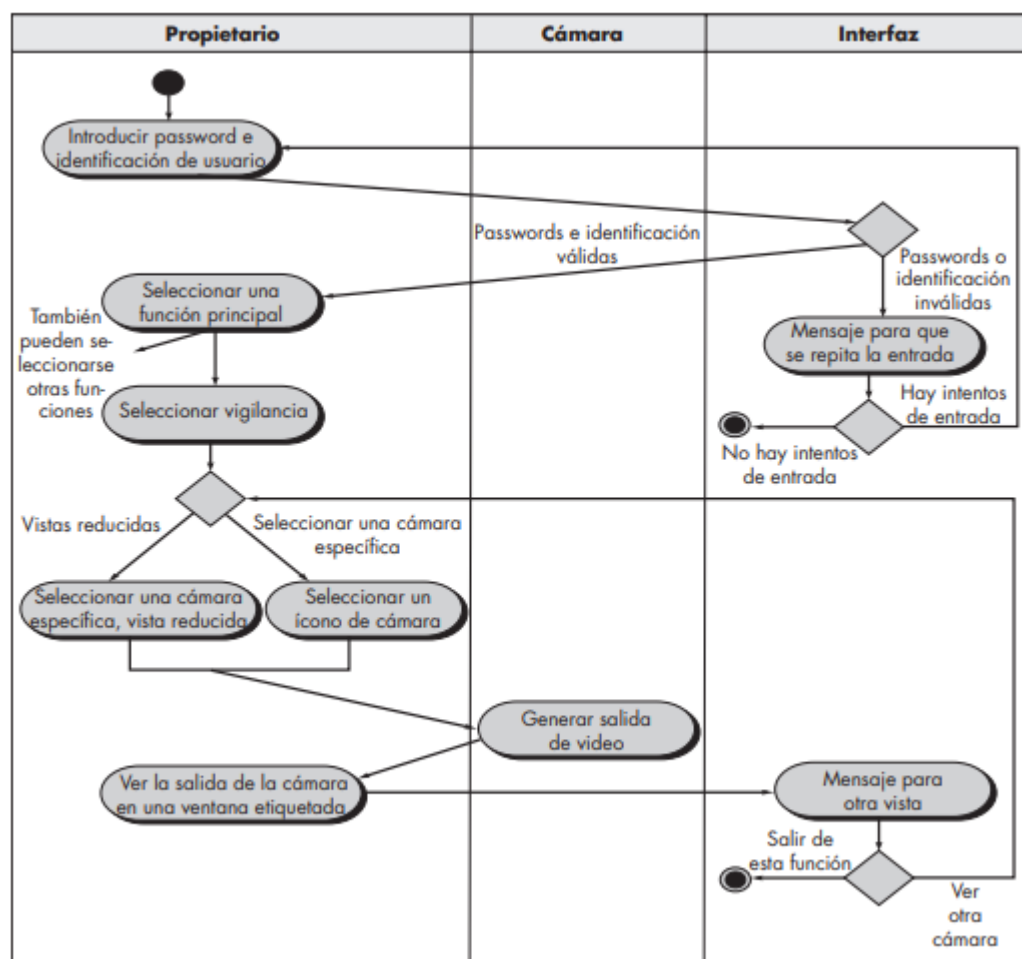


Ilustración 10: Diagrama de canal para la función Acceder a la vigilancia con cámaras por internet, mostrar vistas de cámaras

Fuente: (Pressman, 2010).

La ilustración 10 del diagrama de actividades se ha reorganizado para que las actividades asociadas con una capa de escaneo específica que pertenece al canal de esa clase. Ejemplo: Clase de Interface es la interfaz de usuario

vista por el propietario. Diagrama de las actividades incluyen dos mensajes que son responsabilidad de la interfaz: “mensaje para que se repita la entrada” y “mensaje para otra vista”. Estas notificaciones y decisiones relativas están ubicados en el canal de interfaz. Sin embargo, las flechas se mueven de este canal de regreso al del propietario, donde se producen las acciones del propietario(Pressman, 2010).

Los casos de uso, además de los diagramas de actividades y de canal, están orientados al procedimiento. Representan cómo diferentes actores invocan funciones específicas (u otros pasos de procedimiento) en respuesta a los requisitos del sistema. Pero la visión procedimental de los requisitos es sólo una dimensión del sistema(Pressman, 2010).



Recuerde que. Un diagrama de canal (swimlane) representa el flujo de acciones y decisiones e indica qué actores efectúan cada una de ellas.

4. MODELADO BASADO EN CLASES:

El modelado basado en clases representa los objetos que manipula el sistema y los procesos (también conocidos como métodos o servicios) se aplicarán a objetos para realizar operaciones y relaciones (algunas jerárquicas) entre objetos y para colaborar que ocurre entre clases específicas. Incluye elementos de modelo basados en clases, las clases, objetos, atributos, operaciones y modelos clase-responsabilidad-colaborador (CRC), diagramas y paquetes de Colaboración. En las siguientes secciones, hay una serie de las instrucciones informales que ayudarán a identificarlos y representarlos(Pressman, 2010).

4.1. Identificación de las clases de análisis.

Mirando una habitación, notamos un grupo de objetos físicos que son identificados, categorizados y fácil de identificar (en términos de atributos y operaciones). Pero cuando el problema de implementación del programa se ve, las clases (y los objetos) son más difíciles de diseñar. Comenzamos definiendo las

clases analizando los casos de uso desarrollados como parte del modelo de requisitos y realizando un Análisis gramatical [Abb83] sobre los casos de uso desarrollados del sistema construido. Las clases se determina subrayando cada sustantivo o frase que contiene para ser ingresado en la tabla fácil. Deben tenerse en cuenta los sinónimos, si se requiere la clase (sustantivo) es necesario para ejecutar la solución; por el contrario, si sólo es necesario para la descripción de la solución es parte del espacio del problema(Pressman, 2010).

Pero, ¿qué buscas una vez que has seleccionado todos los sustantivos?

Clases de análisis que aparece de una de las siguientes formas:

- Entidades externas (ej., otros sistemas, equipos y personal) que producen o emplean información que utilizará el sistema informático.
- Cosas (informes, pantallas, cartas, señales, etc.) forman parte del campo de información del problema.
- Ocurrencia o evento (como una transferencia de propiedad o la ejecución de una cadena movimiento del robot) ocurre dentro de la actividad el sistema.
- Roles (gerente, ingeniero, vendedor, etc.) desempeñados por personas interactivas con el sistema
- Unidades organizativas (departamento, grupo, equipo, etc.) relacionadas con un registro.
- Lugares (piso de producción o plataforma de carga) determina el contexto del fallo del sistema y funcionamiento general.
- Las estructuras (sensores, vehículos de cuatro ruedas, ordenadores, etc.) definen una case de objetos o clases relacionadas.

Esta clasificación es sólo una de las muchas propuestas en la literatura. por ejemplo, budd [Bud96] propone categorizar clases que incluyen productores (fuente) y consumidor (sumideros) de datos, administradores de datos, vista, clases de observador y clases de auxiliares. Asimismo, es importante reconocer las clases u objetos que no existen normalmente, la clase no debe tener un nombre de acción determinista [Cas89]. Por ejemplo, si los desarrolladores de software de sistemas de imágenes médicas han identificado un objeto con Nombre InvertirImagen o incluso Inversióndeimagen, cometerán un pequeño error. La imagen obtenida del programa, por supuesto, puede ser una clase (algo que es parte de un dominio de Información). La inversión de la imagen es un proceso que se aplica a un objeto. Puede que esta inversión se defina como una operación del objeto imagen, pero no lo hará como una

clase separada con la connotación “inversión de imagen” como dice Cashman [Cas89]: El propósito del enrutamiento de objetos es contener datos y trabajando en ellos. Para ilustrar cómo se pueden definir las clase analíticas en las primeras etapas de Modelado, considere el análisis de revisión (sustantivos subrayados, verbos en cursiva de una narración de procesamiento de la característica Safe House Security (Pressman, 2010).

La función de seguridad CasaSegura permite que el propietario configure el sistema de seguridad cuando se instala, vigila todos los sensores conectados al sistema de seguridad e interactúa con el propietario a través de internet, una PC o panel de control.

Durante la instalación, la PC de CasaSegura se utiliza para programar y configurar el sistema. Se asigna a cada sensor un número y tipo, se programa un password maestro para activar y desactivar el sistema y se introducen números telefónicos para marcar cuando ocurre un evento de sensor.

Cuando se reconoce un evento de sensor, el software invoca una alarma audible instalada en el sistema. Después de un tiempo de retraso que especifica el propietario durante las actividades de configuración del sistema, el software marca un número telefónico de un servicio de monitoreo, proporciona información acerca de la ubicación y reporta la naturaleza del evento detectado. El número telefónico se vuelve a marcar cada 20 segundos hasta que se obtiene la conexión telefónica.

El propietario recibe información de seguridad a través de un panel de control, la PC o un navegador, lo que en conjunto se llama interfaz. La interfaz despliega mensajes de aviso e información del estado del sistema en el panel de control, la PC o la ventana del navegador. La interacción del propietario tiene la siguiente forma...

Con los sustantivos se proponen varias clases potenciales:

Clase potencial	Clasificación general
propietario	rol de entidad externa
sensor	entidad externa
panel de control	entidad externa
instalación	ocurrencia
sistema (alias sistema de seguridad)	cosa
número, tipo	no objetos, atributos de sensor
password maestro	cosa
número telefónico	cosa
evento de sensor	ocurrencia
alarma audible	entidad externa
servicio de monitoreo	unidad organizacional o entidad externa

Ilustración 11: Función de seguridad CasaSegura, Clases potenciales
Fuente: (Pressman, 2010)

La lista continuará hasta que se hayan considerado todos los nombres de la narrativa de procesamiento. Tenga en cuenta que cada elemento de la lista se denomina objeto potencia. Los lectores deben considerar todo antes de tomar su decisión final. Coad y Yourdon [Coa91] sugieren seis características de selección para usar cuando cada categoría potencial se considera incluida en el modelo analítico: (Pressman, 2010).

1. Información retenida. La categoría potencial solo será útil durante el análisis si es necesario recordar información sobre ella para que el sistema funcione (Pressman, 2010).

2. Servicios necesarios. Una posible clase debe contener un conjunto específico de operaciones que modifiquen el valor de sus propiedades de alguna manera (Pressman, 2010).

3. Atributos múltiples. Durante el análisis de necesidades, debe prestar atención a la información básica; de hecho, una clase con un atributo puede ser útil durante el diseño, pero durante el análisis se puede representar mejor como un atributo de otra clase (Pressman, 2010).

4. Atributos comunes. Para una clase potencial, se define un conjunto de atributos y estas propiedades se aplican a todas las instancias de la clase (Pressman, 2010).

5. Operaciones comunes. Se selecciona un conjunto de actividades para la categoría potencial y estos se aplican a todas las clases (Pressman, 2010).

6. Requerimientos esenciales. Las entidades externas que aparecen en el espacio del problema y crean o consumen la información necesaria para ejecutar cualquier solución del sistema siempre se definirán como clases en el modelo de requerimiento (Pressman, 2010).

Para ser considerado una categoría legítima para incluir en el formulario de solicitud, el objeto potencial debe satisfacer todas (o casi todas) las características anteriores. La resolución de incluir clases potenciales en el modelo analítico es algo subjetiva, y una evaluación posterior puede resultar en la eliminación o inclusión de un objeto. Sin embargo, el primer paso del modelado basado en clases se deriva la definición de éstas y las correspondientes medidas a tomar (incluso subjetivas). Con esto, las características de selección se aplicarán en la lista de clases potenciales de CasaSegura: (Pressman, 2010).

Clase potencial	Número de característica que se aplica
propietario	rechazada: 1 y 2 fallan, aunque la 6 aplica
sensor	aceptada: se aplican todas
panel de control	aceptada: se aplican todas
instalación	rechazada
sistema (alias sistema de seguridad)	aceptada: se aplican todas
número, tipo	rechazada: 3 fallan, atributos de sensores
password maestro	rechazada: 3 falla
número telefónico	rechazada: 3 falla
evento de sensor	aceptada: se aplican todas
alarma audible	aceptada: se aplican 2, 3, 4, 5 y 6
servicio de monitoreo	rechazada: 1 y 2 fallan aunque la 6 aplica

Ilustración 12: Clase potencial y número de características que se aplica
Fuente: (Pressman, 2010).

Cabe señalar que:

- 1) la lista anterior no es exhaustiva. Para completar el formulario, deben agregar capas adicionales
- 2) algunas clases potenciales serán rechazadas y se convertirán en atributos para otras que sí fueron aceptadas (por ejemplo, el número y el tipo son atributos de un sensor, y contraseña maestra y número de teléfono pueden convertirse en atributos del sistema);
- 3) diferentes declaraciones de problemas conducirán a diferentes decisiones de 'aceptar o rechazar' (Por ejemplo, si cada propietario tiene una clave privada o es reconocido por reconocimiento de voz, la clase de propietario cumple con las características 1 y 2 y será aceptada) (Pressman, 2010).

4.2. Especificación de atributos.

Los Atributos que describen una clase elegida para ser incluida en el modelo de requerimientos, son, en esencia, atributos que definen la clase (que muestra lo que significa la clase en el contexto del espacio del problema). Por ejemplo, si construye un sistema de análisis estadístico de jugadores de béisbol profesionales, los atributos

de clase de jugador serán muy diferente de su tipo cuando se utilice en el contexto de un sistema de pensiones deportivas. En la primera parte, atributos como nombre, porcentaje de bateo, porcentaje de fildeo, años jugados y juegos jugados serán relevantes. Para el segundo, la precedencia será importante, pero otros serán reemplazados (o se crearían) atributos como salario promedio, crédito hacia el retiro completo, opciones del plan de pensiones elegido, dirección de correo (Pressman, 2010).

Para desarrollar un conjunto significativo de atributos para una clase analítica, debe estudiar cada caso de uso y seleccionar cosas "razonables" que "pertenezcan" a esa clase. Además, para cada capítulo se debe responder la siguiente pregunta: "¿Cuáles son los aspectos de los datos (compuestos o elementales) definen exactamente esta clase en el contexto del problema en la pregunta?" (Pressman, 2010).

Para mayor claridad, considere la clase de sistema específica para CasaSegura. El propietario configura la función de seguridad para reflejar la información de los sensores, respuesta de alarma, activación o desactivación, identificación, etc. Estos datos agregados se representan como: (Pressman, 2010).

- información de identificación = identificación del sistema + número telefónico de verificación + estado del sistema
- información de respuesta de la alarma = tiempo de retraso + número telefónico
- información de activación o desactivación = password maestro + número de intentos permisibles + password temporal (Pressman, 2010).

Cada elemento de datos a la derecha del signo igual se puede definir con más detalle, hasta el nivel más básico, pero para nuestros propósitos es una lista razonable de propiedades para la clase sistema (parte sombreada de la ilustración 13). Los sensores son parte del sistema CasaSegura, pero no se enumeran como los datos o atributos están en la ilustración 13. El sensor se define como una clase y se asociarán múltiples objetos Sensor con la clase Sistema. En general, se evade establecer algo como atributo si más de uno va a asociarse con la clase (Pressman, 2010).

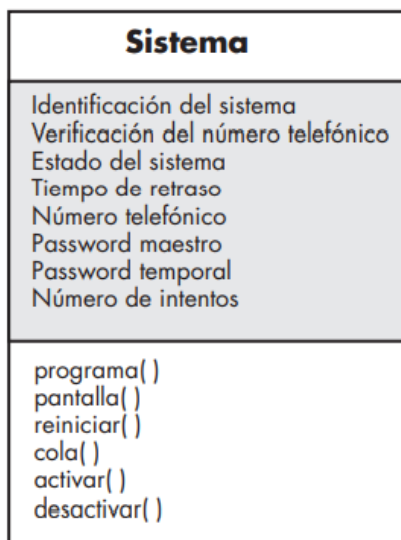


Ilustración 13: Diagrama de clase para la clase sistema
 Fuente: (Pressman, 2010).

4.3. Definición de las operaciones.

Las operaciones determinan el comportamiento de un objeto. Aunque hay muchos tipos diferentes de operaciones, que generalmente se dividen en cuatro categorías principales: (Pressman, 2010).

- 1) operaciones a manipular los datos de alguna manera (por ejemplo, agregar, eliminar, modificar, seleccionar, etc.),
- 2) operaciones aritméticas,
- 3) operaciones que consultan un estado de un objeto y
- 4) la operación que supervisa un objeto para un evento de consola.

Estas funciones se realizan mediante operaciones sobre atributos o sobre asociaciones de estos, por lo tanto, una operación debe tener 'conocimiento' de la naturaleza de los atributos y asociaciones de clase (Pressman, 2010).

Como primera iteración para obtener el conjunto de operaciones de una clase de análisis, se investiga la historia de tratamiento (o caso de uso), y se optan aquellas que correspondan razonablemente a la clase. Para lograr esto, se vuelve a realizar el análisis gramatical y verbos aislados. Algunas de ellas serán operaciones legítimas y pueden asociarse fácilmente con una clase en particular. Por ejemplo, la narración de CasaSegura, donde encontramos que “el número y tipo asignado al sensor” o “la

contraseña maestra programada para activar y desactivar el sistema” se refiere a un número de cosas: (Pressman, 2010).

- La operación de asignar () está relacionada con la clase de sensor.
- La operación del programa () se aplicará a la clase del sistema.
- Activate() y deactivate() son operaciones que se aplican a la clase System.

La operación de programar () probablemente se dividirá en algunas suboperaciones específicos adicionales para la configuración del sistema. Por ejemplo, programar () implica identificar números de teléfono, configurar funciones del sistema (como crear paneles de sensores, ingresar características de alarma, etc.) e ingrese la(s) contraseña(s). Pero actualmente, programar () se define como una sola operación. Además del análisis, obtendrá más información sobre otras operaciones, si se hace esto considere la conexión que se produce entre los objetos. Estas cosas se comunican con la línea de transmisión de mensajes unos a otros (Pressman, 2010).

5. MODELADO CLASE-RESPONSABILIDAD-COLABORADOR (CRC).

El modelado de clase-responsabilidad-colaborador (CRC) [Wir90] proporciona un método simple de definir y organizar los capítulos relacionados con los requisitos de un sistema o producto. Ambler [Amb95] describe el modelo CRC de la siguiente manera: (Pressman, 2010).

“El modelo CRC es en realidad un conjunto de tarjetas índice estándar que representan clases. Las tarjetas se dividen en tres partes. En la parte superior de la tarjeta se escribe el nombre de la clase, las responsabilidades de clase se enumeran en el lado izquierdo del cuerpo y los colaboradores se enumeran en el lado derecho. De hecho, el modelo CRC usa tarjeta índice de una forma real o imaginaria. El objetivo es desarrollar una representación estructurada de las clases. Las responsabilidades son atributos y operaciones asociados con una clase (Pressman, 2010).

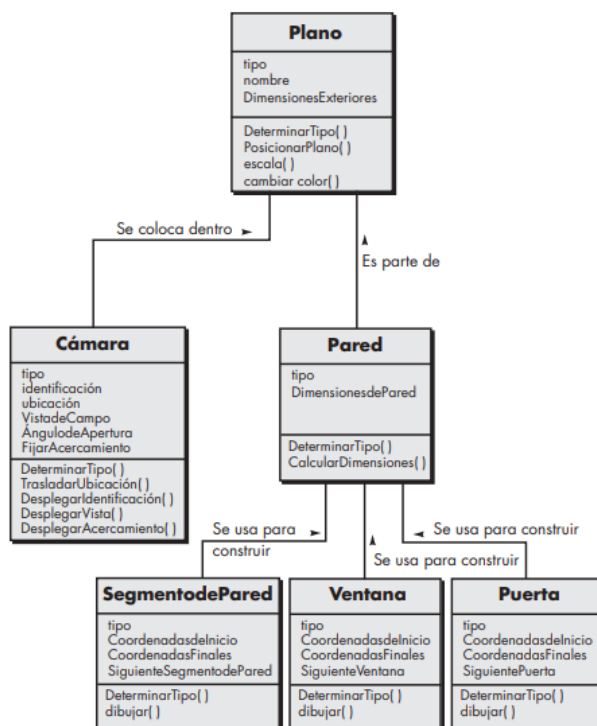


Ilustración 14: Diagrama de clase para Plano (véase el análisis en el recuadro)
 Fuente: (Pressman, 2010).

En otras palabras, la responsabilidad es “cualquier cosa que toda la clase sepa o haga” [Amb95]. Los colaboradores son ciertas clases que debes ofrecer para obtener una capa de información necesaria para cumplir con la responsabilidad. Normalmente la colaboración implica solicitar información o una acción específica (Pressman, 2010).

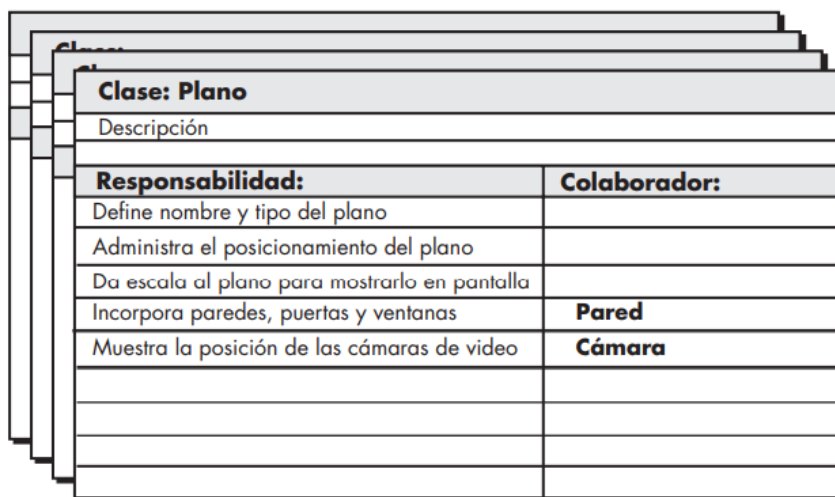


Ilustración 15: Modelo de tarjeta CRC índice IGURA 6.11

Fuente: (Pressman, 2010).

La tarjeta CRC simplificado para la clase de plano se muestra en la ilustración 15:

Plano: La lista de responsabilidades en el modelo CRC es preliminar y se puede agregar o modificar. Las clases Pared y Cámara se clasifican en función de las responsabilidades que se requieren para trabajar juntas (Pressman, 2010).

Clases se presentaron lineamientos básicos para la identificación de clases y objetos. La taxonomía de los tipos de clase se puede ampliar con las siguientes categorías: (Pressman, 2010).

- Las clases de entidad, también conocidas como clases de negocios o modelos, se extraen directamente de la declaración del problema (por ejemplo, plano y sensor). Es común para estas clases representen objetos que están almacenados en la base de datos y que han existido durante un período de tiempo mientras dure la aplicación (a menos que se elimine específicamente) (Pressman, 2010).
- Clases de frontera se utilizan para crear interfaces (como pantallas interactivas o informes impresos) a los que los usuarios se refieren e interactúan cuando usan el software. El objeto entidad contiene información importante para el usuario, pero no aparecen. Las clases de frontera están diseñadas de manera responsable para administrar cómo se muestran los objetos de entidad al usuario. Por ejemplo, una clase de frontera llamada VentanadeCámara será responsable de mostrar la salida de la cámara de vigilancia de CasaSegura (Pressman, 2010).
- Clases de controlador gestionan la Unidad de Trabajo [UML03] de principio a fin. Esto significa que las clases de controlador están diseñadas para manejar 1) crear o actualizar objetos de entidad, 2) instancias de objetos de frontera como recopilar información de objetos de entidad, 3) conexiones complejas entre objetos y 4) validar datos que se comunican entre objetos, usuarios y aplicaciones. En general, las clases de controlador no se consideran hasta que comienza la actividad de diseño (Pressman, 2010).

Responsabilidades. Wirfs-Brock et al. [Wir90] recomienda cinco principios para asignar responsabilidad a las clases:

1. La inteligencia del sistema debe distribuirse entre las clases para adaptarse mejor a las necesidades del problema. Cada aplicación tiene un nivel inteligencia, es decir, lo que el sistema sabe y puede hacer. Esa inteligencia se distribuye entre las clases de diferentes maneras. Las clases tontas (aquellas con pocas responsabilidades) pueden diseñarse para actuar como subordinados de la clase de "inteligentes" (los que tienen mucha responsabilidad). A pesar de que este enfoque hace que el flujo de control fluya hacia un sistema, tiene algunos inconvenientes: concentra toda la inteligencia en unas pocas clases, lo que dificulta el cambio, y tiende a requerir más clases y, por lo tanto, tendrá que trabajar más que el desarrollo. Si la inteligencia del sistema se distribuye uniformemente entre las clases de una aplicación, cada objeto sabe algo y hace solo unas pocas cosas (generalmente bien definidas) y mejorar la coherencia del sistema (Pressman, 2010).

Esto facilita el mantenimiento del software y reduce el impacto de los resultados del cambio. Para determinar si la inteligencia del sistema está correctamente distribuida, las responsabilidades enumeradas en cada formulario CRC deben evaluarse para determinar si la clase tiene una lista muy larga de responsabilidades. Aquí se genera más la concentración de inteligencia, además, la responsabilidad de cada clase debe tener el mismo nivel de abstracción. Por ejemplo, entre las actividades enumeradas para una clase agregada llamada RevisarCuenta, el evaluador enumera dos responsabilidades: hacer el balance de la cuenta y eliminar comprobaciones concluidas. La primera operación (responsabilidad) que involucra un complejo procedimiento matemático y lógico. El segundo es una simple actividad de oficina. Como estas dos operaciones no están en el mismo nivel de abstracción, la eliminación de las comprobaciones completados debería estar bajo la responsabilidad de RevisarEntrada, una clase incluida en la clase agregada RevisarCuenta(Pressman, 2010).

2. Cada responsabilidad debe definirse de la manera más amplia posible. Esta directiva indica que las responsabilidades compartidas (tanto atributos como operaciones) deben estar en la parte superior de la jerarquía de clases (dado que son generales y se aplicará a todas las subclases) (Pressman, 2010).

3. La información y el comportamiento relacionado con ella deben residir dentro de la misma clase. Esto satisface el orientado a objetos llamado encapsulamiento. Los datos y los procesos con los que trabaja deben agruparse en una sola unidad cohesiva (Pressman, 2010).

4. La información sobre una cosa debe colocarse en una clase, no distribuirse en varias clases. Una categoría es responsable de almacenar y procesar cierto tipo de información. En general, esta responsabilidad debe ser común a varias clases. Si se divulga la información, será aún más difícil el mantenimiento y más complejo de probar (Pressman, 2010).

5. Cuando corresponda, las responsabilidades deben compartirse entre las clases relacionadas. Hay muchos casos en los que varios objetos relacionados deben ser el mismo comportamiento al mismo tiempo. Ejemplo de un videojuego, tome las siguientes clases que aparecerán en la pantalla: Jugador, CuerpodelJugador, BrazosdelJugador, PiernasdelJugador y CabezadelJugador. Cada una de estas clases tiene sus propios atributos (como posición, dirección, color y velocidad) y todas deben actualizarse y mostrarse a medida que el usuario manipula el joystick. Por lo tanto, la responsabilidad de actualizar () y desplegar () debe ser compartida por cada uno de los objetos mencionados. El jugador sabe cuándo algo ha cambiado y requiere actualizar (). Funciona con otros objetos para obtener una nueva posición u orientación, pero cada objeto controla su propia pantalla (Pressman, 2010).

Colaboraciones. La clase cumple con sus responsabilidades de una de dos maneras:

- 1) Usa operaciones especiales para manipular sus atributos, cumpliendo así una determinada responsabilidad, o 2) cooperación con otras clases. Wirfs-Brock et al. [Wir90] define la colaboración como: (Pressman, 2010).

La colaboración representa las solicitudes que el cliente envía al servidor para cumplirla responsabilidad. La colaboración es la ejecución de un contrato entre cliente y servidor [...] Se dice que un ser coopera con otro ser si, para cumplir con su responsabilidad, necesita transmitir otro objeto todos los mensajes. La colaboración única ocurre de una manera: que representa una solicitud del cliente al servidor. Desde el punto de vista del cliente, cada cooperación que tiene es relaciona con una responsabilidad particular realizada por el servidor (Pressman, 2010).

La colaboración se define determinando si una clase puede cumplir con cada responsabilidad. De lo contrario, necesita interactuar con otra clase. es cooperación. Por ejemplo, considere la función de seguridad CasaSegura; como parte del proceso de activación, el objeto PaneldeControl debe determinar si están abiertos algunos sensores (Pressman, 2010).

La responsabilidad se define como `determinar-estado-delsensor()`. Si los sensores están abiertos, `paneldeControl` debe establecer el atributo de Estado en No está listo. La Información de los datos del sensor se recopilan de cada objeto del sensor. Por lo tanto, la responsabilidad de `determinar-estadodelsensor()` se lleva a cabo solo si `PaneldeControl` funciona en conjunto con el sensor (Pressman, 2010).

Para ayudar a identificar a los colaboradores, se estudiaron tres correlaciones diferentes. Entre las clases [Wir90]: 1) la relación es parte de, 2) la relación tiene conocimiento de, y 3) la relación depende de (Pressman, 2010).

Todas las clases que forman parte de una clase compuesta conectada a ella por la relación es parte de. Considere las clases definidas por el juego mencionado anteriormente la clase `CuerpodelJugador` es-parte-de `Jugador`, igual que `BrazosdelJugador`, `PiernasdelJugador` y `CabezadelJugador`. En UML, estas relaciones se representan mediante agrupaciones como la ilustración 16. Cuando una clase necesita obtener información de otra clase, se crea una relación tiene-conocimiento-de. La responsabilidad de `determinar-estado-delsensor()` antes mencionada es un ejemplo de esto (Pressman, 2010).

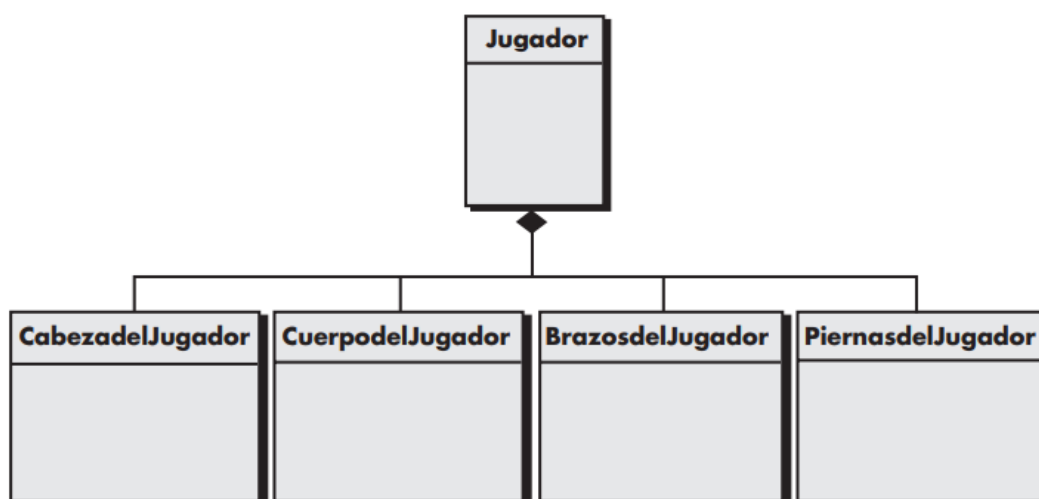


Ilustración 16: Una clase agregada compuesta
 Fuente: (Pressman, 2010).

La relación depende-de significa que hay dos clases que tienen una relación de dependencia indefinida por tiene-conocimiento-de ni por es-parte-de. Ejemplo:

CabezadelJugador siempre debe estar conectada a CuerpodelJugador (a menos que el juego de video sea particularmente violento), pero cada objeto puede existir sin el conocimiento directo del otro. Un atributo del objeto CabezadelJugador, llamado posición-central, se determina a partir de la posición central de CuerpodelJugador. Esta información se obtiene por medio de un tercer objeto, Jugador, que la obtiene de CuerpodelJugador. Entonces, CabezadelJugador depende-de CuerpodelJugador (Pressman, 2010).

En todos los casos, el nombre de la clase de colaboración se registra en el formulario de tarjeta CRC índice, así como la responsabilidad que crea la cooperación. Así, la tarjeta índice contiene una lista de responsabilidades y cooperación que hacen que se cumplan (ver ilustración 15) (Pressman, 2010).

Una vez que se desarrolla un modelo CRC completo, los participantes lo revisan utilizando el siguiente enfoque [Amb95]: (Pressman, 2010).

1. Todos los participantes de la revisión (del modelo CRC) recibieron un subconjunto de modelo de tarjetas índice CRC. Los colaboradores deben estar separados (para que ningún revisor necesite dos tarjetas trabajando juntas) (Pressman, 2010).
2. Todos los escenarios de casos de uso (y los gráficos correspondientes) deben organizarse en dos categorías (Pressman, 2010).
3. El líder ha leído intencionalmente el caso de uso. Cuando llega un objeto nombrado, emitiendo una ficha a la persona al respectivo titular de la tarjeta índice de clase. Por ejemplo, un caso de uso de CasaSegura tiene la siguiente historia: Los propietarios observa el panel de control de CasaSegura para determinar si el sistema está listo para recibir una entrada. Si el sistema no está listo, cerrar físicamente las puertas y ventanas de modo que el indicador listo aparezca [un indicador no está listo implica que un sensor se encuentra abierto, es decir, que una puerta o ventana está abierta]. Cuando el administrador de revisión se acerca al "panel de control" en la historia del caso de uso, entrega la ficha al titular de la tarjeta índice PaneldeControl. La frase 'implícitamente que el sensor está abierto' requiere que la tarjeta índice tenga una responsabilidad que validará la implicación (esto se logra mediante la responsabilidad determinar-estado-delsensor()). Además de responsabilidad,

tarjeta índice hay un Sensor colaborador. Luego, la ficha se pasa al objeto sensor (Pressman, 2010).

4. Una vez que se transfiere la ficha, se le pide al titular de la tarjeta sensor que describa las responsabilidades enumeradas en la tarjeta. El equipo determina si una (o más) responsabilidades cumplen con los requisitos del caso de uso (Pressman, 2010).
5. Si las responsabilidades y colaboraciones mencionadas en las tarjetas índice no corresponden al caso de uso, se revisarán y modificarán. Esto puede incluir la selección de nuevas clases ((y las tarjetas CRC índice correspondientes) o la especificación en las tarjetas existentes de responsabilidades o colaboraciones nuevas o revisadas. Este modo de operación continúa hasta el final del caso de uso. cuando todos los casos de uso se han revisado, el modelado de requisitos continúa (Pressman, 2010).

5.1. Modelo de comportamiento:

La notación del modelo que hemos considerado hasta ahora representa los elementos estáticos del modelo de requisitos. Tiempo de transición al comportamiento dinámico sistema o producto. Para ello, este comportamiento se expresa en función de eventos y tiempos específicos. Los modelos de comportamiento indican cómo responde el programa a los eventos o estímulo externo. Para crear un formulario, siga estos pasos: (Pressman, 2010).

1. Evaluar todos los casos de uso para comprender completamente la secuencia de reacción en el sistema.

2. Identificación de eventos que determinan la secuencia de interacciones e inclusión.

¿Cómo te relacionas con cosas específicas?

3. Cree una secuencia para cada caso de uso.

4. Construya un diagrama de estado para el sistema.

5. Revisar los patrones de comportamiento para verificar su precisión y consistencia.

5.2. Identificar los eventos con el caso de uso

Un evento ocurre cada vez que el sistema y los actores comparten información. También se dice que el evento no es información de intercambio, pero la verdad es que se ha intercambiado información. El caso de uso se estudia con el propósito de intercambiar información. Para mayor claridad, volvamos al caso de uso de una parte de la funcionalidad de seguridad de CasaSegura. El propietario usa el teclado para ingresar una contraseña de cuatro dígitos. Las contraseñas se comparan con una contraseña válida registrada en el sistema. Si la contraseña es incorrecta, panel de control emitirá un pitido y se reiniciará para recibir entradas adicionales. Si la contraseña es correcta, el panel de control estará esperando más acciones. Las partes subrayadas en un escenario de caso de uso se refieren a eventos. Debe especificar un actor para cada evento, anote la información intercambiada y mencione cualquier condición o restricción. Como ejemplo de un evento común, considere la oración subrayada en el caso de uso "El propietario usa el teclado para ingresar una password de cuatro dígitos". como contexto del modelo de los requerimientos, el objeto PropietariodeCasa pasa un evento al objeto ControlPanel. El evento se puede llamar la contraseña ingresada. La información transmitida son los cuatro dígitos que forman la contraseña, pero no son una parte esencial del modelo de comportamiento. Es importante señalar que algunos eventos tienen un claro impacto en el flujo de control de un caso de uso, mientras que otros no. Por ejemplo, el evento de password introducida explícitamente no cambia el flujo de control del caso de uso, sino que cambia el resultado l evento password comparado (derivado de la interacción el "password se compara con el password válido guardado en el sistema") tendrán un efecto explícito en el flujo de información y control del software CasaSegura. Una vez definidos todos los eventos, se asignan a los objetos asociados. Los objetos son responsables de generar eventos (como que el propietario cree eventos para la password introducido) o conocer eventos que ocurrieron en otros lugares (PaneldeControl reconoce el resultado binario del evento de password comparado) (Pressman, 2010).

5.3. Representaciones de estado

En el contexto del modelado del comportamiento, se deben considerar dos características diferentes estados: 1) el estado de cada clase donde el sistema realiza

su función, y 2) el estado del sistema se observa desde el exterior mientras realiza su función (Pressman, 2010).

El estado de la clase tiene propiedades pasivas y activas [Cha93]. El estado pasivo es simplemente el estado actual de todos los atributos del objeto. Ejemplo: Estado pasivo de clase de jugador (en la aplicación de videojuegos) incluirá los atributos actuales posición y orientación del jugador así como otras características relacionadas con el juego (por ejemplo, un tema que incluya deseos mágicos restantes) (Pressman, 2010).

El estado activo del objeto indica que el estado actual del objeto pasa por tratamiento o procesamiento continuo. La clase jugador puede obtener los estados activos: moverse, en descanso, herido, en curación, atrapado, perdido, etc. Debe ocurrir un evento (a veces llamado disparador o trigger) para forzar a un objeto a hacer de un estado activo a otro (Pressman, 2010).

5.4. Diagrama de estado.

El componente del modelo de comportamiento es un diagrama de estado UML, que representa los estados activos para cada clase y los eventos (disparadores) que provocan cambios en los estados activos (Pressman, 2010).

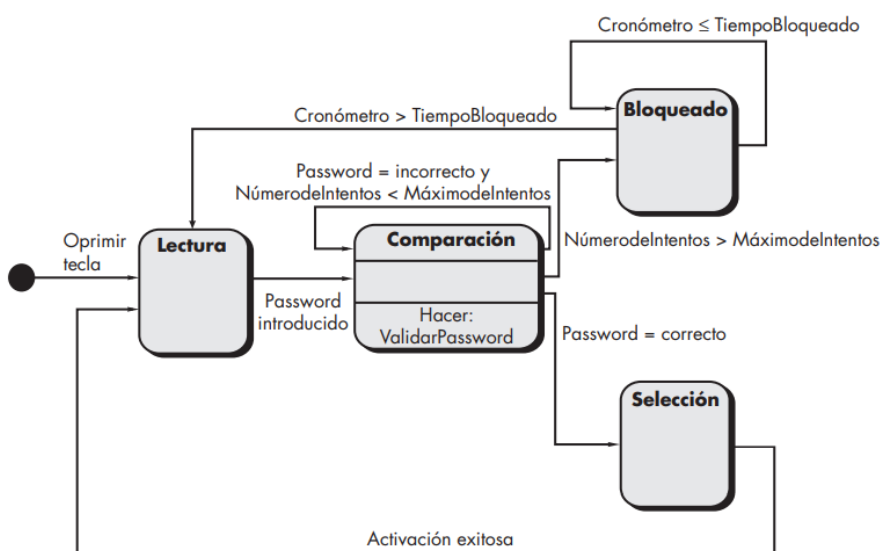


Ilustración 17: Diagrama de estado para la clase PaneldeControl
 Fuente: (Pressman, 2010).

La ilustración 17 muestra el diagrama de estado del objeto PaneldeControl en la función de seguridad CasaSegura. Cada flecha que se muestra en la ilustración 17 representa la transición desde el estado activo de un objeto al otro. El comentario de cada flecha representa el evento que desencadenó la transición. Aunque el modelo de estado activo proporciona información útil sobre la "historia de vida" de un objeto, se puede especificar información adicional para una comprensión más profunda del comportamiento del objeto (Pressman, 2010).

Además de identificar el evento que provocó la transmisión, se puede identificar un guardia y una acción [Cha93]. Guardia es una condición booleana que debe cumplirse para que se produzca la transición. por ejemplo, La guardia para la transición del estado "lectura" al estado "comparación" se determina en la ilustración 17 analizando el caso de uso: (Pressman, 2010).

Si (password de entrada = 4 dígitos) entonces comparar con el password guardado

En general, la guardia de transición depende del valor de una o más atributos del objeto, en otras palabras, depende del estado pasivo del objeto. Una acción ocurre al mismo tiempo que la transición de estado o como resultado de ella y suele incluir una o más operaciones (responsabilidad) del objeto. Por ejemplo, la acción n conectada con el evento password introducido como se aprecia en la ilustración 17, es una operación llamada ValidarPassword() que accede al objeto de la password y realiza una comparación número por número para validar el password introducido (Pressman, 2010).

5.5. Diagrama de secuencia.

El segundo tipo de representación del comportamiento se llama diagrama de secuencia UML, que muestra cómo los eventos causan transformaciones de un objeto a otro. Una vez que haya identificado los objetos a través de un análisis de casos de uso, el modelador crea un diagrama de secuencia: una representación de cómo los eventos hacen que los flujos de uno a otro como función del tiempo. Básicamente, el diagrama de secuencia es una versión taquigráfica del caso de uso. Representa las clases de passwords y los eventos que hacen que el comportamiento evolucione de una clase a otra (Pressman, 2010).

La ilustración 18 muestra un diagrama de secuencia de la función de seguridad de CasaSegura. Cada flecha representa un evento (de un caso de uso) y muestra cómo es un comportamiento interconectado entre los objetos de CasaSegura. El tiempo se mide en la dirección vertical (hacia abajo) y los rectángulos estrechos verticales angostos representan como toma el procesamiento de la actividad. Los estados se muestran a lo largo de una línea de tiempo vertical (Pressman, 2010).

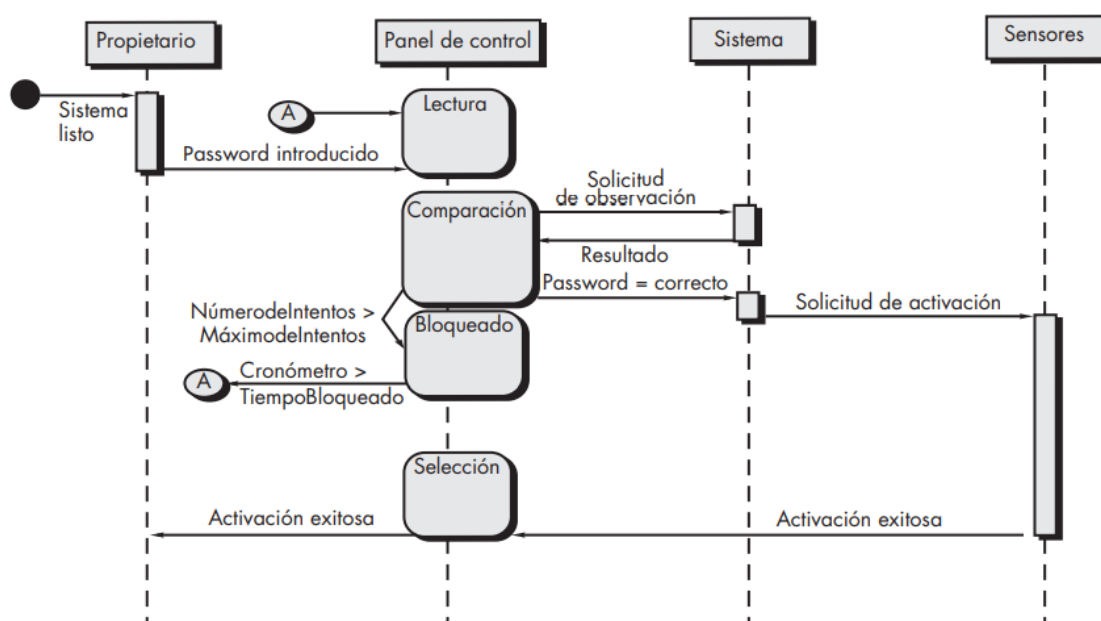


Ilustración 18: Diagrama de secuencia(parcial) para la función CasaSegura
Fuente: (Pressman, 2010).

El primer evento, sistema listo, se origina en el entorno externo y redirige el comportamiento para el objeto propietario. El propietario ingresa la contraseña. solicitud de observación pasa al Sistema, que observa el password en una base de datos sencilla y devuelve un resultado (encontrada o no encontrada) a Panel de control (ahora en el estado de comparación). Una contraseña válida da como resultado una password = correcto hacia Sistema, que activa Sensores con el evento de solicitud de activación. Finalmente, el control regresa al propietario con un evento activación exitosa. Una vez que se elabora un diagrama de secuencia completo, se presentan todos los eventos que causan las transiciones entre los objetos del sistema, se agrupan en un conjunto de eventos de entrada y salida (desde un objeto). Esta información es útil para crear un diseño efectivo para el sistema que se está construyendo (Pressman, 2010).

6. MODELADO DE REQUERIMIENTOS PARA WEBAPPS

Los desarrolladores web a menudo se muestran escépticos cuando surge esta idea de Análisis de necesidades para webapps. A menudo dicen: “Después de todo, el desarrollo web debe ser flexible y el análisis lleva tiempo. nos retrasará cuando necesitamos diseñar y construir una aplicación web.” El análisis de necesidades toma tiempo, pero resolver el problema equivocado toma más tiempo. La pregunta que todo desarrollador web debe responder es simple: ¿Está usted estés seguro de que entiendes los requisitos del problema? Si la respuesta es "sí", Entonces, podría ser posible omitir el modelado de requisitos, pero si la respuesta es No, entonces ésta debe llevarse a cabo (Pressman, 2010).

6.1. ¿Cuánto análisis es suficiente?

La profundidad de los requisitos de modelado para aplicaciones web depende de los siguientes factores: (Pressman, 2010).

- Tamaño y complejidad de una aplicación web.
- Número de participantes (el análisis ayuda a identificar los requisitos conflictivos de diferentes fuentes).
- El tamaño del grupo de aplicaciones web.
- La medida en que los miembros del equipo han trabajado juntos antes (el análisis ayuda a desarrollar un entendimiento común del proyecto).
- El éxito de la organización depende directamente del éxito de webapp.

Lo contrario de los puntos anteriores es que cuanto más pequeño sea el proyecto, mejor con menos participantes, un equipo de desarrollo más cohesionado y menos críticas a la aplicación, tiene sentido adoptar un enfoque de análisis más ligero. Aunque es buena idea analizar el problema antes de empezar a diseñar, no lo es que cualquier análisis debe preceder a cualquier percepción. De hecho, diseñe una parte específica para una aplicación web que solo requiera solicitudes de análisis que solo

afecten a esta parte de la Aplicación web. Como ejemplo de CasaSegura, la estética general de un sitio web (diseños, colores, etc.) se puede diseñar correctamente sin necesidad de analizar los requisitos funcionales de las capacidades de comercio electrónico. Basta con analizar esta parte de las preguntas relacionadas con el trabajo de diseño adicional que se asignará (Pressman, 2010).

6.2. Entrada del modelado de los requerimientos

Al diseñar webapp este proceso incluye una actividad de comunicación que identifica a las partes interesadas y los tipos de usuarios, el contexto comercial, que definen la información y los objetivos de la aplicación, los requisitos generales de la webapp y los escenarios de uso, y la información que se convierte en entrada para el modelado de requisitos. Esta información se presenta como una descripción en lenguaje natural, en un sentido amplio las características, gráficos y otras formas de representación informal (Pressman, 2010).

El análisis toma esta información, la construye utilizando un esquema de representación definido formalmente (si lo hay) y luego crea modelos de salida más precisos. El modelo de requisitos proporciona una indicación detallada de la verdadera estructura del problema y proporciona una idea de cómo podría ser la solución. En el sexto capítulo, se presenta la función AVC-MVC (cámara de vigilancia). en este momento esta función parece relativamente simple y se describe en detalle como parte del caso de uso. Sin embargo, los casos de uso pueden revelar información oculta, oscuro o poco claro. Algunos aspectos de esta información faltante aparecerán automáticamente durante el proceso de diseño. Los ejemplos pueden incluir el diseño específico y la apariencia de los botones de función y percepción estética, tamaños de planos, ángulos de cámara y planos de las casas, o incluso detalles como longitudes máximas y mínimas de la casa. Algunos de estos aspectos son decisiones de diseño (como la apariencia del botón) y otros requisitos (como la longitud de la clave) que no afectan significativamente el trabajo de diseño original. Pero parte de la información faltante puede afectar el diseño general y estar más relacionada con comprensión real de los requisitos. Por ejemplo: (Pressman, 2010).

P1: ¿Cuál es la resolución del video de salida que dan las cámaras de CasaSegura?

P2: ¿Qué ocurre si se encuentra una condición de alarma mientras la cámara está siendo vigilada?

P3: ¿Cómo maneja el sistema las cámaras con vistas panorámicas y de acercamiento?

P4: ¿Qué información debe darse junto con la vista de la cámara (por ejemplo, ubicación, fecha y hora, último acceso, etcétera)?

Ninguno de estos problemas fue identificado o considerado durante el desarrollo inicial del caso de uso; sin embargo, las respuestas pueden tener un gran impacto en diferentes aspectos del diseño, por lo tanto, es razonable concluir que incluso si la conexión proporciona una base de comprensión, el análisis requiere el fortalecimiento de esta comprensión a través de más explicación. Como la estructura del problema se delinea como parte del modelo de requerimientos, invariablemente surgen preguntas. Son éstas las que llenan los huecos y, en ciertos casos, en realidad ayudan a encontrarlos (Pressman, 2010).

En resumen, la información obtenida durante la actividad de comunicación será la entrada al modelo de los requerimientos, cualquiera que sea, desde un correo electrónico informal hasta un proyecto detallado con escenarios de uso exhaustivos y especificaciones del producto (Pressman, 2010).

6.3. Salida del modelado de los requerimientos

El análisis de requisitos proporciona un mecanismo disciplinado para la representación y evaluación del contenido y funcionamiento de la webapp, las formas de interacción que encontrarán los usuarios, y el entorno e infraestructura en la que se encuentra la webapp (Pressman, 2010).

Cada una de estas características está representada por un conjunto de modelos permisibles. Las solicitudes de webapp se analizan de manera ordenada. Aunque los modelos específicos dependen en gran medida de la naturaleza de la webapp, existen cinco categorías principales de ellos (Pressman, 2010).

- Modelo de contenido: Define el conjunto de contenido que servirá a la webapp. El contenido incluye texto, gráficos, imágenes, video y datos de audio.
- Modelo de interacción: describe cómo el usuario interactúa con él webapp.

- Modelo funcional: define los procesos que se aplicarán al contenido de la webapp y
- Describir las funciones de procesamiento independientemente de otros contenidos, pero necesario para el usuario final.
- Modelo de navegación: define la estrategia general de navegación de la webapp.
- Modelo de configuración: describe el entorno y la infraestructura en los que se encuentra la configuración la webapp (Pressman, 2010).

Cada uno de estos modelos se puede desarrollar utilizando un esquema de representación (a menudo denominado "lenguaje") que permite que el propósito y la estructura se comuniquen y evalúen fácilmente entre los miembros del equipo de ingeniería web y otros participantes. Así que aquí hay una lista de los principales problemas (por ejemplo, errores, omisiones, discrepancias, sugerencias de mejoras o cambios, puntos de aclaración, etc.) para solucionar el problema (Pressman, 2010).

6.4. Modelo del contenido de las webapps

El modelo de contenido incluye elementos estructurales que dan una perspectiva importante de la solicitud de contenido de la webapp. Estos elementos estructurales incluyen objetos de contenido y todas las clases analíticas, entidades visibles para el usuario crean o manipulan al interactuar con una webapp (Pressman, 2010).

El contenido se puede desarrollar antes de que se publique la webapp, mientras se construye o cuando ya operé. En todos los casos, se combina mediante una referencia de navegación en la estructura general de una webapp. El objeto de contenido es la descripción del producto en forma de texto, un artículo que describe un evento deportivo, una foto tomada durante el evento deportivo, comentarios de los usuarios en el foro de análisis, logotipo animado de empresa, un clip corto de un discurso o una grabación de audio de una presentación con toboganes. Los objetos de contenido pueden almacenarse como archivos separados, incrustarse directamente en páginas web o recuperarse dinámicamente de una base de datos. En otras palabras, un objeto de contenido es información coherente proporcionada al usuario final. Las audiencias de contenido se determinan directamente a partir de casos de uso, por estudio, estudiando las descripciones de escenarios incluyendo referencias directas e indirectas al contenido. por ejemplo, Una webapp que soporta CasaSegura creada en CasaSeguraAsegurada.com. Un caso de uso, comprar

componentes seleccionados de CasaSegura, describe la situación deseada para comprar un componente CasaSegura y obtener la siguiente oración: (Pressman, 2010).

Podré obtener información descriptiva y de precios de cada componente del producto. El modelo de contenido debe poder describir el objeto de contenido del componente. En muchos casos, para establecer requisitos para el contenido que debe diseñarse y una vez implementado, debería ser suficiente una lista simple de objetos de contenido, junto con una breve descripción de cada uno de ellos. Sin embargo, en algunos casos, el modelo de contenido aprovecha el análisis más rico ilustra gráficamente las relaciones entre los objetos de contenido y la jerarquía mantenida por la webapp. Por ejemplo, considere el árbol de datos [Sri01] generado por el componente CasaSeguraAsegurada.com que se muestra en la ilustración 19. Un árbol representa una jerarquía de información utilizada para describir un elemento. Aspectos de datos simples o complejos (uno o más valores de datos) representados por rectángulos sin sombrear. Objetos de contenido se representa mediante rectángulos sombreados. En la foto, se establece la descripción con cinco objetos (rectángulos sombreados). En algunos casos, uno o más de estos objetos mejorará a medida que crezca el árbol de datos. Se puede crear un árbol de datos para cualquier contenido, incluido el número de objetos de contenido y aspectos de datos. El árbol de datos fue desarrollado para este propósito para definir las relaciones jerárquicas entre los objetos de contenido y proporciona un método de verificación de contenido para que las deficiencias e inconsistencias se descubran antes de que comience el diseño. Además, el árbol de datos sirve como base para diseñar el contenido (Pressman, 2010).

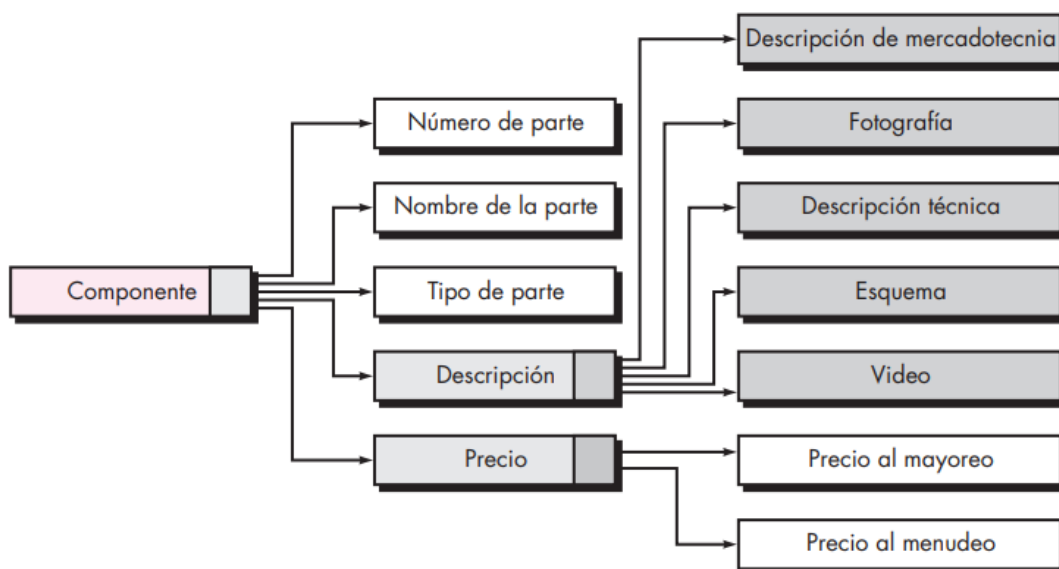


Ilustración 19: Árbol de datos para el componente Casa SeguraAsegurada.com

Fuente (Pressman, 2010).

6.5. Modelo de la interacción para webapps

La mayoría de las webapps permiten una "conversación" entre el usuario final y la funcionalidad, el contenido y el comportamiento de la aplicación. Esta conversación se describe usando un modelo de interacción que consta de uno o más de los siguientes elementos: (Pressman, 2010).

- 1) Casos de Uso,
- 2) Diagrama de secuencia,
- 3) Diagrama de estado
- 4) Prototipo de interfaz de usuario

En muchos casos, un conjunto de casos de uso es suficiente para describir la interacción en Nivel de análisis (se introducirán más mejoras y detalles durante el proceso de diseño). pero, cuando la secuencia de interacciones sea compleja e implique algún tipo de análisis o tareas múltiples, debe ilustrarse más de cerca con un diagrama. La forma de la interfaz de usuario, el contenido que proporciona, los mecanismos de interacción que implementa y la estética general de las comunicaciones entre el usuario y la webapp tiene mucho que ver con la satisfacción del cliente y el éxito general del programa. Aunque la creación de prototipos de interfaz de usuario se declara como una actividad de diseño, se debe hacer esto al

crear un modelo analítico editando lo más rápido posible la representación física de la interfaz de usuario, Entre más pronto se revise la representación física de la interfaz de usuario, más probable es que los consumidores finales obtengan lo que desean (Pressman, 2010).

Dado que existen muchas herramientas para crear webapp baratas y potentes en su funcionalidad, es mejor construir un prototipo de interfaz con el empleo de ellas. El prototipo implementará los principales enlaces de navegación y mostrará muy bien la pantalla general similar al que se va a construir. Por ejemplo, si necesita estar disponible para el usuario final las cinco funciones principales del sistema, el prototipo debe representarlas como verán cuando accede a la webapp por primera vez. ¿Se darán vínculos gráficos? ¿Dónde se desplegará el menú de navegación? ¿Qué otra información verá el usuario? Preguntas como éstas son las que debe responder el prototipo (Pressman, 2010).

6.6. Modelo funcional para las webapps

Muchas webapps brindan una variedad de funciones computacionales y de procesamiento directamente relacionadas con el contenido porque lo utilizan o porque lo producen). A menudo son un objetivo importante para la interacción entre el usuario y la webapp (Pressman, 2010).

Por esta razón, los requisitos funcionales deben analizarse y modelarse si es necesario. Un modelo funcional encuentra dos elementos de procesamiento de la webapp, cada uno representa un nivel de abstracción procedimental: 1) Funciones observables por los usuarios que la webapp pone a su disposición, y 2) las operaciones que se encuentran en clases de Análisis de los comportamientos asociados con la clase (Pressman, 2010).

Funciones observables por el usuario agregan todas las funciones de procesamiento directamente por el usuario. Por ejemplo, una webapp financiera puede implementar muchas funciones financieras (como una calculadora de ahorro de matrícula o fondo de jubilación). Estas funciones ya son realizadas por operaciones dentro de las clases de análisis, pero desde la perspectiva del usuario final; Mostrar resultados es la función (o más bien los datos proporcionados por la función) (Pressman, 2010).

En un nivel más bajo de abstracción del procedimiento, se describe un modelo de requisitos donde el procesamiento se realiza mediante operaciones de clases de análisis.

Estas operaciones manipulan los atributos de la clase e involucran a las clases que cooperan entre sí para lograr algún comportamiento deseado. Independientemente del nivel de abstracción del procedimiento, el diagrama de actividad UML es utilizado para representar sus detalles. A nivel de análisis, diagrama de actividad solo deben usarse cuando la función es relativamente compleja. Gran parte de la complejidad de webapps no radican en la funcionalidad que proporcionan, sino en la naturaleza de la aplicación de la información que se puede acceder y cómo se manipula (Pressman, 2010).

6.7. Modelos de configuración para las webapps

En algunos casos, la plantilla de configuración es solo una lista de atributos del lado del servidor y del cliente. Sin embargo, para webapps más complejas, existen algunas dificultades de configuración (por ejemplo, equilibrio de carga entre varios servidores, arquitectura de caché, bases de datos remotas, diferentes servidores que sirven varios objetos en el mismo servidor del sitio web, etc.) que afectan el análisis y diseño de impacto (Pressman, 2010).

El diagrama de despliegue UML utilizado en situaciones en las que se deben considerar arquitecturas de configuración complejas. Para CasaSeguraAsegurada.com, el contenido general y las características deben especificarse para que se pueda acceder a ellos a través de las principales aplicaciones web (como las con 1% o más de participación de mercado). Por el contrario, las restricciones son aceptables respecto a las funciones de seguimiento y control más complejas (que sólo es accesible para los usuarios tipo Propietario) o para un grupo más pequeño de clientes. La modelo de configuración de CasaSeguraAsegurada.com también definirá un proceso cruzado con su base de datos de productos y aplicaciones de vigilancia (Pressman, 2010).

BIBLIOGRAFÍA

- Kendall, K., & Kendall, J. (2011). *Análisis y Diseños de Sistemas 8va Edición* - ISBN: 9786073205771. http://cotana.informatica.edu.bo/downloads/Id-Analisis-y-Disenio-de-Sistemas_Kendall-8va.pdf
- M. Gómez, J. Cervantes, & P. González. (2019). Fundamentos de Ingeniería de Software. In *Jurnal Ekonomi Malaysia* (Vol. 51, Issue 2).
- Pressman, R. S. (2010). *Ingeniería del Software Un enfoque práctico* (Séptima Ed). McGraw-Hill. <http://cotana.informatica.edu.bo/downloads/Id-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF>